DATA MINING COURSE READINGS

# Chapter 04: Classification · Naïve Bayes, SVM, ANNs & Mode

Curated Textbook Readings from Tan et al. · Matching Lecture: IE500_DM_04_Classification_Part2.pdf

**REFERENCE TEXTBOOK**

**Introduction to Data Mining (2nd Edition, Global Edition)**

Authors: Pang-Ning Tan, Michael Steinbach, Anuj Karpatne, Vipin Kumar | Pearson, 2019

Curated by: Ehsan Shahbazi | Shahid Beheshti University (SBU)

## INCLUDED CHAPTER SECTIONS & TOPIC SCOPE

- **Section 6.4: Naïve Bayes Classifier** — Book p. 415·430
  Bayes theorem, prior/conditional probabilities, continuous Gaussian, Laplace smoothing
- **Section 6.7: Artificial Neural Networks (ANN)** — Book p. 452·465
  Perceptron, activation functions, multilayer perceptron (MLP), backpropagation algorithm
- **Section 6.8: Deep Learning** — Book p. 465·478
  Deep networks, Convolutional Neural Networks (CNN), regularization, initialization
- **Section 6.9: Support Vector Machines (SVM)** — Book p. 479·498
  Max-margin hyperplane, soft-margin SVM, slack variables, nonlinear kernels
- **Sections 3.5, 3.7: Model Selection & Hyperparameters** — Book p. 177·184, 189·192
  Holdout, k-fold cross-validation, validation sets, grid/random search, nested cross-validation

**STUDENT STUDY GUIDE & SCOPE**

Includes probabilistic classification (Naïve Bayes), kernel methods (SVM), Artificial Neural Networks and Deep Learning, alongside rigorous model selection and hyperparameter optimization techniques (Nested CV).

8. Nearest neighbor classifiers can produce wrong predictions unless the appropriate proximity measure and data preprocessing steps are taken. For example, suppose we want to classify a group of people based on attributes such as height (measured in meters) and weight (measured in pounds). The height attribute has a low variability, ranging from 1.5 m to 1.85 m, whereas the weight attribute may vary from 90 lb. to 250 lb. If the scale of the attributes are not taken into consideration, the proximity measure may be dominated by differences in the weights of a person.

## 6.4 Naïve Bayes Classifier

Many classification problems involve uncertainty. First, the observed attributes and class labels may be unreliable due to imperfections in the measurement process, e.g., due to the limited preciseness of sensor devices. Second, the set of attributes chosen for classification may not be fully representative of the target class, resulting in uncertain predictions. To illustrate this, consider the problem of predicting a person's risk for heart disease based on a model that uses their diet and workout frequency as attributes. Although most people who eat healthily and exercise regularly have less chance of developing heart disease, they may still be at risk due to other *latent factors*, such as heredity, excessive smoking, and alcohol abuse, that are not captured in the model. Third, a classification model learned over a finite training set may not be able to fully capture the true relationships in the overall data, as discussed in the context of model overfitting in Chapter 3. Finally, uncertainty in predictions may arise due to the inherent *random nature* of real-world systems, such as those encountered in weather forecasting problems.

In the presence of uncertainty, there is a need to not only make predictions of class labels but also provide a measure of confidence associated with every prediction. Probability theory offers a systematic way for quantifying and manipulating uncertainty in data, and thus, is an appealing framework for assessing the confidence of predictions. Classification models that make use of probability theory to represent the relationship between attributes and class labels are known as **probabilistic classification models**. In this section, we present the naïve Bayes classifier, which is one of the simplest and most widely-used probabilistic classification models.

### 6.4.1 Basics of Probability Theory

Before we discuss how the naïve Bayes classifier works, we first introduce some basics of probability theory that will be useful in understanding the probabilistic classification models presented in this chapter. This involves defining the notion of probability and introducing some common approaches for manipulating probability values.

Consider a variable $X$, which can take any discrete value from the set $\{x_1, \ldots, x_k\}$. When we have multiple observations of that variable, such as in a data set where the variable describes some characteristic of data objects, then we can compute the relative frequency with which each value occurs. Specifically, suppose that $X$ has the value $x_i$ for $n_i$ data objects. The relative frequency with which we observe the event $X = x_i$ is then $n_i/N$, where $N$ denotes the total number of occurrences ($N = \sum_{i=1}^{k} n_i$). These relative frequencies characterize the uncertainty that we have with respect to what value $X$ may take for an unseen observation and motivates the notion of probability.

More formally, the **probability** of an event $e$, e.g., $P(X = x_i)$, measures how likely it is for the event $e$ to occur. The most traditional view of probability is based on relative frequency of events (frequentist), while the Bayesian viewpoint (described later) takes a more flexible view of probabilities. In either case, a probability is always a number between 0 and 1. Further, the sum of probability values of all possible events, e.g., outcomes of a variable $X$ is equal to 1. Variables that have probabilities associated with each possible outcome (values) are known as **random variables**.

Now, let us consider two random variables, $X$ and $Y$, that can each take $k$ discrete values. Let $n_{ij}$ be the number of times we observe $X = x_i$ and $Y = y_j$, out of a total number of $N$ occurrences. The **joint probability** of observing $X = x_i$ and $Y = y_j$ together can be estimated as

$$P(X = x_i, Y = y_i) = \frac{n_{ij}}{N}. \tag{6.7}$$

(This is an estimate since we typically have only a finite subset of all possible observations.) Joint probabilities can be used to answer questions such as "what is the probability that there will be a surprise quiz today `AND` I will be late for the class." Joint probabilities are symmetric, i.e., $P(X = x, Y = y) = P(Y = y, X = x)$. For joint probabilities, it is to useful to consider their sum with respect to one of the random variables, as described in the following equation:

$$\sum_{j=1}^{k} P(X = x_i, Y = y_j) = \frac{\sum_{j=1}^{k} n_{ij}}{N} = \frac{n_i}{N} = P(X = x_i), \tag{6.8}$$

where $n_i$ is the total number of times we observe $X = x_i$ irrespective of the value of $Y$. Notice that $n_i/N$ is essentially the probability of observing $X = x_i$. Hence, by summing out the joint probabilities with respect to a random variable $Y$, we obtain the probability of observing the remaining variable $X$. This operation is called **marginalization** and the probability value $P(X = x_i)$ obtained by marginalizing out $Y$ is sometimes called the **marginal probability** of $X$. As we will see later, joint probability and marginal probability form the basic building blocks of a number of probabilistic classification models discussed in this chapter.

Notice that in the previous discussions, we used $P(X = x_i)$ to denote the probability of a particular outcome of a random variable $X$. This notation can easily become cumbersome when a number of random variables are involved. Hence, in the remainder of this section, we will use $P(X)$ to denote the probability of any generic outcome of the random variable $X$, while $P(x_i)$ will be used to represent the probability of the specific outcome $x_i$.

### Bayes Theorem

> *Suppose you have invited two of your friends Alex and Martha to a dinner party. You know that Alex attends* 40% *of the parties he is invited to. Further, if Alex is going to a party, there is an* 80% *chance of Martha coming along. On the other hand, if Alex is not going to the party, the chance of Martha coming to the party is reduced to* 30%*. If Martha has responded that she will be coming to your party, what is the probability that Alex will also be coming?*

Bayes theorem presents the statistical principle for answering questions like the previous one, where evidence from multiple sources has to be combined with prior beliefs to arrive at predictions. Bayes theorem can be briefly described as follows.

Let $P(Y|X)$ denote the **conditional probability** of observing the random variable $Y$ whenever the random variable $X$ takes a particular value. $P(Y|X)$ is often read as the probability of observing $Y$ *conditioned* on the outcome of $X$. Conditional probabilities can be used for answering questions such as "given that it is going to rain today, what will be the probability that I will go to the class." Conditional probabilities of $X$ and $Y$ are related to their joint

probability in the following way:

$$P(Y|X) = \frac{P(X,Y)}{P(X)}, \quad \text{which implies} \tag{6.9}$$

$$\begin{aligned} P(X,Y) &= P(Y|X) \times P(X) \\ &= P(X|Y) \times P(Y). \end{aligned} \tag{6.10}$$

Rearranging the last two expressions in Equation 6.10 leads to Equation 6.11, which is known as **Bayes theorem**:

$$P(Y|X) = \frac{P(X|Y)P(Y)}{P(X)}. \tag{6.11}$$

Bayes theorem provides a relationship between the conditional probabilities $P(Y|X)$ and $P(X|Y)$. Note that the denominator in Equation 6.11 involves the marginal probability of $X$, which can also be represented as

$$P(X) = \sum_{i=1}^{k} P(X, y_i) = \sum_{i=1}^{k} P(X|y_i) \times P(y_i).$$

Using the previous expression for $P(X)$, we can obtain the following equation for $P(Y|X)$ solely in terms of $P(X|Y)$ and $P(Y)$:

$$P(Y|X) = \frac{P(X|Y)P(Y)}{\sum_{i=1}^{k} P(X|y_i)P(y_i)}. \tag{6.12}$$

**Example 6.4. [Bayes Theorem]** Bayes theorem can be used to solve a number of inferential questions about random variables. For example, consider the problem stated at the beginning on inferring whether Alex will come to the party. Let $P(A = 1)$ denote the probability of Alex going to a party, while $P(A = 0)$ denotes the probability of him not going to a party. We know that

$$\begin{aligned} & P(A = 1) = 0.4, \\ \text{and} \quad & P(A = 0) = 1 - P(A = 1) = 0.6. \end{aligned}$$

Further, let $P(M = 1|A)$ denote the conditional probability of Martha going to a party conditioned on whether Alex is going to the party. $P(M = 1|A)$ takes the following values:

$$\begin{aligned} & P(M = 1|A = 1) = 0.8, \\ \text{and} \quad & P(M = 1|A = 0) = 0.3. \end{aligned}$$

We can use the above values of $P(M|A)$ and $P(A)$ to compute the probability of Alex going to the party given Martha is going to the party, $P(A = 1|M = 1)$, as follows:

$$\begin{aligned} P(A = 1|M = 1) &= \frac{P(M = 1|A = 1)P(A = 1)}{P(M = 1|A = 0)P(A = 0) + P(M = 1|A = 1)P(A = 1)}, \\ &= \frac{0.8 \times 0.4}{0.3 \times 0.6 + 0.8 \times 0.4} \\ &= \frac{0.32}{0.5} = 0.64. \end{aligned} \tag{6.13}$$

Notice that even though the prior probability $P(A)$ of Alex going to the party is low, the observation that Martha is going, $M = 1$, affects the conditional probability $P(A = 1|M = 1)$. This shows the value of Bayes theorem in combining prior assumptions with observed outcomes to make predictions. Since $P(A = 1|M = 1) > 0.5$, it is more likely for Alex to join if Martha is going to the party.

■

### Using Bayes Theorem for Classification

For the purpose of classification, we are interested in computing the probability of observing a class label $y$ for a data instance given its set of attribute values $\mathbf{x}$. This can be represented as $P(y|\mathbf{x})$, which is known as the **posterior probability** of the target class. Using the Bayes Theorem, we can represent the posterior probability as

$$P(y|\mathbf{x}) = \frac{P(\mathbf{x}|y)P(y)}{P(\mathbf{x})} \tag{6.14}$$

Note that the numerator of the previous equation involves two terms, $P(\mathbf{x}|y)$ and $P(y)$, both of which contribute to the posterior probability $P(y|\mathbf{x})$. We describe both of these terms in the following.

The first term $P(\mathbf{x}|y)$ is known as the **class-conditional** probability of the attributes given the class label. $P(\mathbf{x}|y)$ measures the likelihood of observing $\mathbf{x}$ from the distribution of instances belonging to $y$. If $\mathbf{x}$ indeed belongs to class $y$, then we should expect $P(\mathbf{x}|y)$ to be high. From this point of view, the use of class-conditional probabilities attempts to capture the process from which the data instances were generated. Because of this interpretation, probabilistic

classification models that involve computing class-conditional probabilities are known as **generative classification models**. Apart from their use in computing posterior probabilities and making predictions, class-conditional probabilities also provide insights about the underlying mechanism behind the generation of attribute values.

The second term in the numerator of Equation 6.14 is the **prior probability** $P(y)$. The prior probability captures our prior beliefs about the distribution of class labels, independent of the observed attribute values. (This is the Bayesian viewpoint.) For example, we may have a prior belief that the likelihood of any person to suffer from a heart disease is $\alpha$, irrespective of their diagnosis reports. The prior probability can either be obtained using expert knowledge, or inferred from historical distribution of class labels.

The denominator in Equation 6.14 involves the probability of evidence, $P(\mathbf{x})$. Note that this term does not depend on the class label and thus can be treated as a normalization constant in the computation of posterior probabilities. Further, the value of $P(\mathbf{x})$ can be calculated as $P(\mathbf{x}) = \sum_i P(\mathbf{x}|y_i)P(y_i)$.

Bayes theorem provides a convenient way to combine our prior beliefs with the likelihood of obtaining the observed attribute values. During the training phase, we are required to learn the parameters for $P(y)$ and $P(\mathbf{x}|y)$. The prior probability $P(y)$ can be easily estimated from the training set by computing the fraction of training instances that belong to each class. To compute the class-conditional probabilities, one approach is to consider the fraction of training instances of a given class for every possible combination of attribute values. For example, suppose that there are two attributes $X_1$ and $X_2$ that can each take a discrete value from $c_1$ to $c_k$. Let $n^0$ denote the number of training instances belonging to class 0, out of which $n^0_{ij}$ number of training instances have $X_1 = c_i$ and $X_2 = c_j$. The class-conditional probability can then be given as

$$P(X_1 = c_i, X_2 = c_j|Y = 0) = \frac{n^0_{ij}}{n^0}.$$

This approach can easily become computationally prohibitive as the number of attributes increase, due to the exponential growth in the number of attribute value combinations. For example, if every attribute can take $k$ discrete values, then the number of attribute value combinations is equal to $k^d$, where $d$ is the number of attributes. The large number of attribute value combinations can also result in poor estimates of class-conditional probabilities, since every combination will have fewer training instances when the size of training set is small.

In the following, we present the naïve Bayes classifier, which makes a simplifying assumption about the class-conditional probabilities, known as the

**naïve Bayes assumption**. The use of this assumption significantly helps in obtaining reliable estimates of class-conditional probabilities, even when the number of attributes are large.

### 6.4.2 Naïve Bayes Assumption

The naïve Bayes classifier assumes that the class-conditional probability of all attributes $\mathbf{x}$ can be factored as a product of class-conditional probabilities of every attribute $x_i$, as described in the following equation:

$$P(\mathbf{x}|y) = \prod_{i=1}^{d} P(x_i|y), \tag{6.15}$$

where every data instance $\mathbf{x}$ consists of $d$ attributes, $\{x_1, x_2, \ldots, x_d\}$. The basic assumption behind the previous equation is that the attribute values $x_i$ are **conditionally independent** of each other, given the class label $y$. This means that the attributes are influenced only by the target class and if we know the class label, then we can consider the attributes to be independent of each other. The concept of conditional independence can be formally stated as follows.

#### Conditional Independence

Let $\mathbf{X_1}$, $\mathbf{X_2}$, and $\mathbf{Y}$ denote three sets of random variables. The variables in $\mathbf{X_1}$ are said to be conditionally independent of $\mathbf{X_2}$, given $\mathbf{Y}$, if the following condition holds:

$$P(\mathbf{X_1}|\mathbf{X_2}, \mathbf{Y}) = P(\mathbf{X_1}|\mathbf{Y}). \tag{6.16}$$

This means that conditioned on $\mathbf{Y}$, the distribution of $\mathbf{X_1}$ is not influenced by the outcomes of $\mathbf{X_2}$, and hence is conditionally independent of $X_2$. To illustrate the notion of conditional independence, consider the relationship between a person's arm length ($\mathbf{X_1}$) and his or her reading skills ($\mathbf{X_2}$). One might observe that people with longer arms tend to have higher levels of reading skills, and thus consider $\mathbf{X_1}$ and $\mathbf{X_2}$ to be related to each other. However, this relationship can be explained by another factor, which is the age of the person ($\mathbf{Y}$). A young child tends to have short arms and lacks the reading skills of an adult. If the age of a person is fixed, then the observed relationship between arm length and reading skills disappears. Thus, we can conclude that arm length and reading skills are not directly related to each other and are conditionally independent when the age variable is fixed.

Another way of describing conditional independence is to consider the joint conditional probability, $P(\mathbf{X_1}, \mathbf{X_2}|\mathbf{Y})$, as follows:

$$\begin{aligned} P(\mathbf{X_1}, \mathbf{X_2}|\mathbf{Y}) &= \frac{P(\mathbf{X_1}, \mathbf{X_2}, \mathbf{Y})}{P(\mathbf{Y})} \\ &= \frac{P(\mathbf{X_1}, \mathbf{X_2}, \mathbf{Y})}{P(\mathbf{X_2}, \mathbf{Y})} \times \frac{P(\mathbf{X_2}, \mathbf{Y})}{P(\mathbf{Y})} \\ &= P(\mathbf{X_1}|\mathbf{X_2}, \mathbf{Y}) \times P(\mathbf{X_2}|\mathbf{Y}) \\ &= P(\mathbf{X_1}|\mathbf{Y}) \times P(\mathbf{X_2}|\mathbf{Y}), \end{aligned} \tag{6.17}$$

where Equation 6.16 was used to obtain the last line of Equation 6.17. The previous description of conditional independence is quite useful from an operational perspective. It states that the joint conditional probability of $\mathbf{X_1}$ and $\mathbf{X_2}$ given $Y$ can be factored as the product of conditional probabilities of $\mathbf{X_1}$ and $\mathbf{X_2}$ considered separately. This forms the basis of the naïve Bayes assumption stated in Equation 6.15.

### How a Naïve Bayes Classifier Works

Using the naïve Bayes assumption, we only need to estimate the conditional probability of each $x_i$ given $Y$ separately, instead of computing the class-conditional probability for every combination of attribute values. For example, if $n_i^0$ and $n_j^0$ denote the number of training instances belonging to class 0 with $X_1 = c_i$ and $X_2 = c_j$, respectively, then the class-conditional probability can be estimated as

$$P(X_1 = c_i, X_2 = c_j|Y = 0) = \frac{n_i^0}{n^0} \times \frac{n_j^0}{n^0}.$$

In the previous equation, we only need to count the number of training instances for every one of the $k$ values of an attribute $X$, irrespective of the values of other attributes. Hence, the number of parameters needed to learn class-conditional probabilities is reduced from $d^k$ to $dk$. This greatly simplifies the expression for the class-conditional probability and makes it more amenable to learning parameters and making predictions, even in high-dimensional settings.

The naïve Bayes classifier computes the posterior probability for a test instance $\mathbf{x}$ by using the following equation:

$$P(y|\mathbf{x}) = \frac{P(y) \prod_{i=1}^{d} P(x_i|y)}{P(\mathbf{x})}. \tag{6.18}$$

Since $P(\mathbf{x})$ is fixed for every $y$ and only acts as a normalizing constant to ensure that $P(y|\mathbf{x}) \in [0, 1]$, we can write

$$P(y|\mathbf{x}) \propto P(y) \prod_{i=1}^{d} P(x_i|y).$$

Hence, it is sufficient to choose the class that maximizes $P(y) \prod_{i=1}^{d} P(x_i|y)$.

One of the useful properties of the naïve Bayes classifier is that it can easily work with incomplete information about data instances, when only a subset of attributes are observed at every instance. For example, if we only observe $p$ out of $d$ attributes at a data instance, then we can still compute $P(y) \prod_{i=1}^{p} P(x_i|y)$ using those $p$ attributes and choose the class with the maximum value. The naïve Bayes classifier can thus naturally handle missing values in test instances. In fact, in the extreme case where no attributes are observed, we can still use the prior probability $P(y)$ as an estimate of the posterior probability. As we observe more attributes, we can keep refining the posterior probability to better reflect the likelihood of observing the data instance.

In the next two subsections, we describe several approaches for estimating the conditional probabilities $P(x_i|y)$ for categorical and continuous attributes from the training set.

### Estimating Conditional Probabilities for Categorical Attributes

For a categorical attribute $X_i$, the conditional probability $P(X_i = c|y)$ is estimated according to the fraction of training instances in class $y$ where $X_i$ takes on a particular categorical value $c$.

$$P(X_i = c|y) = \frac{n_c}{n},$$

where $n$ is the number of training instances belonging to class $y$, out of which $n_c$ number of instances have $X_i = c$. For example, in the training set given in Figure 6.8, seven people have the class label `Defaulted Borrower=No`, out of which three people have `Home Owner=Yes` while the remaining four have `Home Owner=No`. As a result, the conditional probability for $P$(`Home Owner=Yes`| `Defaulted Borrower=No`) is equal to 3/7. Similarly, the conditional probability for defaulted borrowers with `Marital Status = Single` is given by $P$(`Marital Status = Single`|`Defaulted Borrower=Yes`) = 2/3. Note that the sum of conditional probabilities over all possible outcomes of $X_i$ is equal to one, i.e., $\sum_c P(X_i = c|y) = 1$.

| | binary | categorical | continuous | class |
|---|---|---|---|---|
| Tid | Home Owner | Marital Status | Annual Income | Defaulted Borrower |
| 1 | Yes | Single | 125K | No |
| 2 | No | Married | 100K | No |
| 3 | No | Single | 70K | No |
| 4 | Yes | Married | 120K | No |
| 5 | No | Divorced | 95K | Yes |
| 6 | No | Married | 60K | No |
| 7 | Yes | Divorced | 220K | No |
| 8 | No | Single | 85K | Yes |
| 9 | No | Married | 75K | No |
| 10 | No | Single | 90K | Yes |

**Figure 6.8.** Training set for predicting the loan default problem.

### Estimating Conditional Probabilities for Continuous Attributes

There are two ways to estimate the class-conditional probabilities for continuous attributes:

1. We can discretize each continuous attribute and then replace the continuous values with their corresponding discrete intervals. This approach transforms the continuous attributes into ordinal attributes, and the simple method described previously for computing the conditional probabilities of categorical attributes can be employed. Note that the estimation error of this method depends on the discretization strategy (as described in Section 2.3.6 on page 83), as well as the number of discrete intervals. If the number of intervals is too large, every interval may have an insufficient number of training instances to provide a reliable estimate of $P(X_i|Y)$. On the other hand, if the number of intervals is too small, then the discretization process may loose information about the true distribution of continuous values, and thus result in poor predictions.

2. We can assume a certain form of probability distribution for the continuous variable and estimate the parameters of the distribution using the training data. For example, we can use a Gaussian distribution to represent the conditional probability of continuous attributes. The Gaussian distribution is characterized by two parameters, the mean, $\mu$, and the variance, $\sigma^2$. For each class $y_j$, the class-conditional probability

for attribute $X_i$ is

$$P(X_i = x_i|Y = y_j) = \frac{1}{\sqrt{2\pi}\sigma_{ij}} \exp\left[-\frac{(x_i - \mu_{ij})^2}{2\sigma_{ij}^2}\right]. \tag{6.19}$$

The parameter $\mu_{ij}$ can be estimated using the sample mean of $X_i$ ($\overline{x}$) for all training instances that belong to $y_j$. Similarly, $\sigma_{ij}^2$ can be estimated from the sample variance ($s^2$) of such training instances. For example, consider the annual income attribute shown in Figure 6.8. The sample mean and variance for this attribute with respect to the class `No` are

$$\overline{x} = \frac{125 + 100 + 70 + \ldots + 75}{7} = 110$$
$$s^2 = \frac{(125 - 110)^2 + (100 - 110)^2 + \ldots + (75 - 110)^2}{6} = 2975$$
$$s = \sqrt{2975} = 54.54.$$

Given a test instance with taxable income equal to \$120K, we can use the following value as its conditional probability given class `No`:

$$P(\texttt{Income=120}|\texttt{No}) = \frac{1}{\sqrt{2\pi}(54.54)} \exp^{-\frac{(120-110)^2}{2\times 2975}} = 0.0072.$$

**Example 6.5. [Naïve Bayes Classifier]** Consider the data set shown in Figure 6.9(a), where the target class is `Defaulted Borrower`, which can take two values `Yes` and `No`. We can compute the class-conditional probability for each categorical attribute and the sample mean and variance for the continuous attribute, as summarized in Figure 6.9(b).

We are interested in predicting the class label of a test instance $\mathbf{x}$ = (`Home Owner`=`No`, `Marital Status` = `Married`, `Annual Income` = \$120K). To do this, we first compute the prior probabilities by counting the number of training instances belonging to every class. We thus obtain $P$(`Yes`) = 0.3 and $P$(`No`) =

| Tid | Home Owner | Marital Status | Annual Income | Defaulted Borrower |
|---|---|---|---|---|
| 1 | Yes | Single | 125K | **No** |
| 2 | No | Married | 100K | **No** |
| 3 | No | Single | 70K | **No** |
| 4 | Yes | Married | 120K | **No** |
| 5 | No | Divorced | 95K | **Yes** |
| 6 | No | Married | 60K | **No** |
| 7 | Yes | Divorced | 220K | **No** |
| 8 | No | Single | 85K | **Yes** |
| 9 | No | Married | 75K | **No** |
| 10 | No | Single | 90K | **Yes** |

(a)

P(Home Owner=Yes|No) = 3/7
P(Home Owner=No|No) = 4/7
P(Home Owner=Yes|Yes) = 0
P(Home Owner=No|Yes) = 1
P(Marital Status=Single|No) = 2/7
P(Marital Status=Divorced|No) = 1/7
P(Marital Status=Married|No) = 4/7
P(Marital Status=Single|Yes) = 2/3
P(Marital Status=Divorced|Yes) = 1/3
P(Marital Status=Married|Yes) = 0

For Annual Income:
If class=No: sample mean=110
sample variance=2975
If class=Yes: sample mean=90
sample variance=25

(b)

**Figure 6.9.** The naïve Bayes classifier for the loan classification problem.

0.7. Next, we can compute the class-conditional probability as follows:

$$\begin{aligned} P(\mathbf{x}|\texttt{No}) &= P(\texttt{Home Owner} = \texttt{No}|\texttt{No}) \times P(\texttt{Status} = \texttt{Married}|\texttt{No}) \\ &\quad \times P(\texttt{Annual Income} = \$120\text{K}|\texttt{No}) \\ &= 4/7 \times 4/7 \times 0.0072 = 0.0024. \end{aligned}$$

$$\begin{aligned} P(\mathbf{x}|\texttt{Yes}) &= P(\texttt{Home Owner} = \texttt{No}|\texttt{Yes}) \times P(\texttt{Status} = \texttt{Married}|\texttt{Yes}) \\ &\quad \times P(\texttt{Annual Income} = \$120\text{K}|\texttt{Yes}) \\ &= 1 \times 0 \times 1.2 \times 10^{-9} = 0. \end{aligned}$$

Notice that the class-conditional probability for class `Yes` has become 0 because there are no instances belonging to class `Yes` with `Status` = `Married` in the training set. Using these class-conditional probabilities, we can estimate the posterior probabilities as

$$\begin{aligned} P(\texttt{No}|\mathbf{x}) &= \frac{0.7 \times 0.0024}{P(\mathbf{x})}. \\ &= 0.0016\alpha. \\ P(\texttt{Yes}|\mathbf{x}) &= \frac{0.3 \times 0}{P(\mathbf{x})}. \\ &= 0. \end{aligned}$$

where $\alpha = 1/P(\mathbf{X})$ is a normalizing constant. Since $P(\texttt{No}|\mathbf{x}) > P(\texttt{Yes}|\mathbf{x})$, the instance is classified as `No`. ■

### Handling Zero Conditional Probabilities

The preceding example illustrates a potential problem with using the naïve Bayes assumption in estimating class-conditional probabilities. If the conditional probability for any of the attributes is zero, then the entire expression for the class-conditional probability becomes zero. Note that zero conditional probabilities arise when the number of training instances is small and the number of possible values of an attribute is large. In such cases, it may happen that a combination of attribute values and class labels are never observed, resulting in a zero conditional probability.

In a more extreme case, if the training instances do not cover some combinations of attribute values and class labels, then we may not be able to even classify some of the test instances. For example, if $P(\texttt{Marital Status} = \texttt{Divorced}|\texttt{No})$ is zero instead of $1/7$, then a data instance with attribute set $\mathbf{x} = (\texttt{Home Owner} = \texttt{Yes}, \texttt{Marital Status} = \texttt{Divorced}, \texttt{Income} = \$120\text{K})$ has the following class-conditional probabilities:

$$P(\mathbf{x}|\texttt{No}) = 3/7 \times 0 \times 0.0072 = 0.$$
$$P(\mathbf{x}|\texttt{Yes}) = 0 \times 1/3 \times 1.2 \times 10^{-9} = 0.$$

Since both the class-conditional probabilities are 0, the naïve Bayes classifier will not be able to classify the instance. To address this problem, it is important to adjust the conditional probability estimates so that they are not as brittle as simply using fractions of training instances. This can be achieved by using the following alternate estimates of conditional probability:

$$\text{Laplace estimate:} \quad P(X_i = c|y) = \frac{n_c + 1}{n + v}, \tag{6.20}$$

$$\text{m-estimate:} \quad P(X_i = c|y) = \frac{n_c + mp}{n + m}, \tag{6.21}$$

where $n$ is the number of training instances belonging to class $y$, $n_c$ is the number of training instances with $X_i = c$ and $Y = y$, $v$ is the total number of attribute values that $X_i$ can take, $p$ is some initial estimate of $P(X_i = c|y)$ that is known a priori, and $m$ is a hyper-parameter that indicates our confidence in using $p$ when the fraction of training instances is too brittle. Note that even

if $n_c = 0$, both Laplace and m-estimate provide non-zero values of conditional probabilities. Hence, they avoid the problem of vanishing class-conditional probabilities and thus generally provide more robust estimates of posterior probabilities.

### Characteristics of Naïve Bayes Classifiers

1. Naïve Bayes classifiers are probabilistic classification models that are able to quantify the uncertainty in predictions by providing posterior probability estimates. They are also generative classification models as they treat the target class as the causative factor for generating the data instances. Hence, apart from computing posterior probabilities, naïve Bayes classifiers also attempt to capture the underlying mechanism behind the generation of data instances belonging to every class. They are thus useful for gaining predictive as well as descriptive insights.

2. By using the naïve Bayes assumption, they can easily compute class-conditional probabilities even in high-dimensional settings, provided that the attributes are conditionally independent of each other given the class labels. This property makes naïve Bayes classifier a simple and effective classification technique that is commonly used in diverse application problems, such as text classification.

3. Naïve Bayes classifiers are robust to isolated noise points because such points are not able to significantly impact the conditional probability estimates, as they are often averaged out during training.

4. Naïve Bayes classifiers can handle missing values in the training set by ignoring the missing values of every attribute while computing its conditional probability estimates. Further, naïve Bayes classifiers can effectively handle missing values in a test instance, by using only the non-missing attribute values while computing posterior probabilities. If the frequency of missing values for a particular attribute value depends on class label, then this approach will not accurately estimate posterior probabilities.

5. Naïve Bayes classifiers are robust to irrelevant attributes. If $X_i$ is an irrelevant attribute, then $P(X_i|Y)$ becomes almost uniformly distributed for every class $y$. The class-conditional probabilities for every class thus receive similar contributions of $P(X_i|Y)$, resulting in negligible impact on the posterior probability estimates.

6. Correlated attributes can degrade the performance of naïve Bayes classifiers because the naïve Bayes assumption of conditional independence no longer holds for such attributes. For example, consider the following probabilities:

$$P(A=0|Y=0)=0.4, \quad P(A=1|Y=0)=0.6,$$
$$P(A=0|Y=1)=0.6, \quad P(A=1|Y=1)=0.4,$$

where $A$ is a binary attribute and $Y$ is a binary class variable. Suppose there is another binary attribute $B$ that is perfectly correlated with $A$ when $Y=0$, but is independent of $A$ when $Y=1$. For simplicity, assume that the conditional probabilities for $B$ are the same as for $A$. Given an instance with attributes $A=0, B=0$, and assuming conditional independence, we can compute its posterior probabilities as follows:

$$\begin{aligned} P(Y=0|A=0,B=0) &= \frac{P(A=0|Y=0)P(B=0|Y=0)P(Y=0)}{P(A=0,B=0)} \\ &= \frac{0.16\times P(Y=0)}{P(A=0,B=0)}. \end{aligned}$$

$$\begin{aligned} P(Y=1|A=0,B=0) &= \frac{P(A=0|Y=1)P(B=0|Y=1)P(Y=1)}{P(A=0,B=0)} \\ &= \frac{0.36\times P(Y=1)}{P(A=0,B=0)}. \end{aligned}$$

If $P(Y=0)=P(Y=1)$, then the naïve Bayes classifier would assign the instance to class 1. However, the truth is,

$$P(A=0,B=0|Y=0)=P(A=0|Y=0)=0.4,$$

because $A$ and $B$ are perfectly correlated when $Y=0$. As a result, the posterior probability for $Y=0$ is

$$\begin{aligned} P(Y=0|A=0,B=0) &= \frac{P(A=0,B=0|Y=0)P(Y=0)}{P(A=0,B=0)} \\ &= \frac{0.4\times P(Y=0)}{P(A=0,B=0)}, \end{aligned}$$

which is larger than that for $Y=1$. The instance should have been classified as class 0. Hence, the naïve Bayes classifier can produce incorrect

results when the attributes are not conditionally independent given the class labels. Naïve Bayes classifiers are thus not well-suited for handling redundant or interacting attributes.

## 6.5 Bayesian Networks

The conditional independence assumption made by naïve Bayes classifiers may seem too rigid, especially for classification problems where the attributes are dependent on each other even after conditioning on the class labels. We thus need an approach to relax the naïve Bayes assumption so that we can capture more generic representations of conditional independence among attributes.

In this section, we present a flexible framework for modeling probabilistic relationships between attributes and class labels, known as **Bayesian Networks**. By building on concepts from probability theory and graph theory, Bayesian networks are able to capture more generic forms of conditional independence using simple schematic representations. They also provide the necessary computational structure to perform inferences over random variables in an efficient way. In the following, we first describe the basic representation of a Bayesian network, and then discuss methods for performing inference and learning model parameters in the context of classification.

### 6.5.1 Graphical Representation

Bayesian networks belong to a broader family of models for capturing probabilistic relationships among random variables, known as **probabilistic graphical models**. The basic concept behind these models is to use graphical representations where the nodes of the graph correspond to random variables and the edges between the nodes express probabilistic relationships. Figures 6.10(a) and 6.10(b) show examples of probabilistic graphical models using directed edges (with arrows) and undirected edges (without arrows), respectively. Directed graphical models are also known as Bayesian networks while undirected graphical models are known as **Markov random fields**. The two approaches use different semantics for expressing relationships among random variables and are thus useful in different contexts. In the following, we briefly describe Bayesian networks that are useful in the context of classification.

A Bayesian network (also referred to as a **belief network**) involves directed edges between nodes, where every edge represents a direction of influence among random variables. For example, Figure 6.10(a) shows a Bayesian network where variable $C$ depends upon the values of variables $A$ and $B$, as indicated by the arrows pointing toward $C$ from $A$ and $B$. Consequently,

sum of all the attributes. If there are missing values in a training instance, it can be discarded from the training set. However, if there are missing values in a test instance, then logistic regression would fail to predict its class label.

## 6.7 Artificial Neural Network (ANN)

Artificial neural networks (ANN) are powerful classification models that are able to learn highly complex and nonlinear decision boundaries purely from the data. They have gained widespread acceptance in several applications such as vision, speech, and language processing, where they have been repeatedly shown to outperform other classification models (and in some cases even human performance). Historically, the study of artificial neural networks was inspired by attempts to emulate biological neural systems. The human brain consists primarily of nerve cells called **neurons**, linked together with other neurons via strands of fiber called **axons**. Whenever a neuron is stimulated (e.g., in response to a stimuli), it transmits nerve activations via axons to other neurons. The receptor neurons collect these nerve activations using structures called **dendrites**, which are extensions from the cell body of the neuron. The strength of the contact point between a dendrite and an axon, known as a **synapse**, determines the connectivity between neurons. Neuroscientists have discovered that the human brain learns by changing the strength of the synaptic connection between neurons upon repeated stimulation by the same impulse.

The human brain consists of approximately 100 billion neurons that are inter-connected in complex ways, making it possible for us to learn new tasks and perform regular activities. Note that a single neuron only performs a simple modular function, which is to respond to the nerve activations coming from sender neurons connected at its dendrite, and transmit its activation to receptor neurons via axons. However, it is the composition of these simple functions that together is able to express complex functions. This idea is at the basis of constructing artificial neural networks.

Analogous to the structure of a human brain, an artificial neural network is composed of a number of processing units, called nodes, that are connected with each other via directed links. The nodes correspond to neurons that perform the basic units of computation, while the directed links correspond to connections between neurons, consisting of axons and dendrites. Further, the weight of a directed link between two neurons represents the strength of the synaptic connection between neurons. As in biological neural systems, the

primary objective of ANN is to adapt the weights of the links until they fit the input-output relationships of the underlying data.

The basic motivation behind using an ANN model is to extract useful features from the original attributes that are most relevant for classification. Traditionally, feature extraction has been achieved by using dimensionality reduction techniques such as PCA (introduced in Chapter 2), which show limited success in extracting nonlinear features, or by using hand-crafted features provided by domain experts. By using a complex combination of inter-connected nodes, ANN models are able to extract much richer sets of features, resulting in good classification performance. Moreover, ANN models provide a natural way of representing features at multiple levels of abstraction, where complex features are seen as compositions of simpler features. In many classification problems, modeling such a hierarchy of features turns out to be very useful. For example, in order to detect a human face in an image, we can first identify low-level features such as sharp edges with different gradients and orientations. These features can then be combined to identify facial parts such as eyes, nose, ears, and lips. Finally, an appropriate arrangement of facial parts can be used to correctly identify a human face. ANN models provide a powerful architecture to represent a hierarchical abstraction of features, from lower levels of abstraction (e.g., edges) to higher levels (e.g., facial parts).

Artificial neural networks have had a long history of developments spanning over five decades of research. Although classical models of ANN suffered from several challenges that hindered progress for a long time, they have re-emerged with widespread popularity because of a number of recent developments in the last decade, collectively known as **deep learning**. In this section, we examine classical approaches for learning ANN models, starting from the simplest model called **perceptrons** to more complex architectures called **multi-layer neural networks**. In the next section, we discuss some of the recent advancements in the area of ANN that have made it possible to effectively learn modern ANN models with deep architectures.

### 6.7.1 Perceptron

A perceptron is a basic type of ANN model that involves two types of nodes: input nodes, which are used to represent the input attributes, and an output node, which is used to represent the model output. Figure 6.20 illustrates the basic architecture of a perceptron that takes three input attributes, $x_1$, $x_2$, and $x_3$, and produces a binary output $y$. The input node corresponding to an attribute $x_i$ is connected via a weighted link $w_i$ to the output node. The

weighted link is used to emulate the strength of a synaptic connection between neurons.

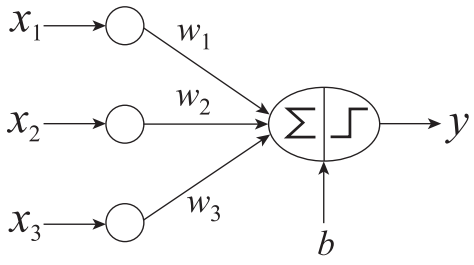



**Figure 6.20.** Basic architecture of a perceptron.

The output node is a mathematical device that computes a weighted sum of its inputs, adds a bias factor $b$ to the sum, and then examines the sign of the result to produce the output $\hat{y}$ as follows:

$$\hat{3}y \quad = \quad \begin{cases} 1, & \text{if } \mathbf{w}^T\mathbf{x} + b > 0. \\ -1, & \text{otherwise.} \end{cases} \tag{6.48}$$

To simplify notations, $\mathbf{w}$ and $b$ can be concatenated to form $\tilde{\mathbf{w}} = (\mathbf{w}^T \; b)^T$, while $\mathbf{x}$ can be appended with 1 at the end to form $\tilde{\mathbf{x}} = (\mathbf{x}^T \; 1)^T$. The output of the perceptron $\hat{y}$ can then be written:

$$\hat{y} = sign(\tilde{\mathbf{w}}^T\tilde{\mathbf{x}}),$$

where the sign function acts as an **activation function** by providing an output value of $+1$ if the argument is positive and $-1$ if its argument is negative.

### Learning the Perceptron

Given a training set, we are interested in learning parameters $\tilde{\mathbf{w}}$ such that $\hat{y}$ closely resembles the true $y$ of training instances. This is achieved by using the perceptron learning algorithm given in Algorithm 6.3. The key computation for this algorithm is the iterative weight update formula given in Step 8 of the algorithm:

$$w_j^{(k+1)} = w_j^{(k)} + \lambda\big(y_i - \hat{y}_i^{(k)}\big)x_{ij}, \tag{6.49}$$

where $w^{(k)}$ is the weight parameter associated with the $i^{th}$ input link after the $k^{th}$ iteration, $\lambda$ is a parameter known as the **learning rate**, and $x_{ij}$ is the value of the $j^{th}$ attribute of the training example $\mathbf{x}_i$. The justification for Equation 6.49 is rather intuitive. Note that $(y_i - \hat{y}_i)$ captures the discrepancy between $y_i$ and $\hat{y}_i$, such that its value is 0 only when the true label and the predicted output match. Assume $x_{ij}$ is positive. If $\hat{y} = 0$ and $y = 1$, then $w_j$ is increased at the next iteration so that $\tilde{\mathbf{w}}^T\mathbf{x}_i$ can become positive. On the other hand, if $\hat{y} = 1$ and $y = 0$, then $w_j$ is decreased so that $\tilde{\mathbf{w}}^T\mathbf{x}_i$ can become negative. Hence, the weights are modified at every iteration to reduce the discrepancies between $\hat{y}$ and $y$ across all training instances. The learning rate $\lambda$, a parameter whose value is between 0 and 1, can be used to control the amount of adjustments made in each iteration. The algorithm halts when the average number of discrepancies are smaller than a threshold $\gamma$.

**Algorithm 6.3** Perceptron learning algorithm.

1: Let $D.train = \{(\tilde{\mathbf{x}}_i, y_i) \mid i = 1, 2, \ldots, n\}$ be the set of training instances.
2: Set $k \leftarrow 0$.
3: Initialize the weight vector $\tilde{\mathbf{w}}^{(0)}$ with random values.
4: **repeat**
5:  **for** each training instance $(\tilde{\mathbf{x}}_i, y_i) \in D.train$ **do**
6:  Compute the predicted output $\hat{y}_i^{(k)}$ using $\tilde{\mathbf{w}}^{(k)}$.
7:  **for** each weight component $w_j$ **do**
8:  Update the weight, $w_j^{(k+1)} = w_j^{(k)} + \lambda(y_i - \hat{y}_i^{(k)})x_{ij}$.
9:  **end for**
10:  Update $k \leftarrow k + 1$.
11:  **end for**
12: **until** $\sum_{i=1}^{n} |y_i - \hat{y_i}^{(k)}|/n$ is less than a threshold $\gamma$

The perceptron is a simple classification model that is designed to learn linear decision boundaries in the attribute space. Figure 6.21 shows the decision boundary obtained by applying the perceptron learning algorithm to the data set provided on the left of the figure. However, note that there can be multiple decision boundaries that can separate the two classes, and the perceptron arbitrarily learns one of these boundaries depending on the random initial values of parameters. (The selection of the optimal decision boundary is a problem that will be revisited in the context of support vector machines in Section 6.9.) Further, the perceptron learning algorithm is only guaranteed to converge when the classes are linearly separable. However, if the classes are not linearly separable, the algorithm fails to converge. Figure 6.22 shows

| $X_1$ | $X_2$ | $X_3$ | Y |
|---|---|---|---|
| 1 | 0 | 0 | -1 |
| 1 | 0 | 1 | 1 |
| 1 | 1 | 0 | 1 |
| 1 | 1 | 1 | 1 |
| 0 | 0 | 1 | -1 |
| 0 | 1 | 0 | -1 |
| 0 | 1 | 1 | 1 |
| 0 | 0 | 0 | -1 |

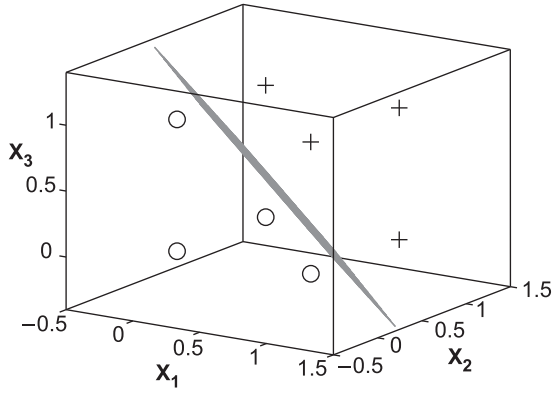



**Figure 6.21.** Perceptron decision boundary for the data given on the left (+ represents a positively labeled instance while o represents a negatively labeled instance.

an example of a nonlinearly separable data given by the XOR function. The perceptron cannot find the right solution for this data because there is no linear decision boundary that can perfectly separate the training instances. Thus, the stopping condition at line 12 of Algorithm 6.3 would never be met and hence, the perceptron learning algorithm would fail to converge. This is a major limitation of perceptrons since real-world classification problems often involve nonlinearly separable classes.

| $X_1$ | $X_2$ | y |
|---|---|---|
| 0 | 0 | −1 |
| 1 | 0 | 1 |
| 0 | 1 | 1 |
| 1 | 1 | −1 |

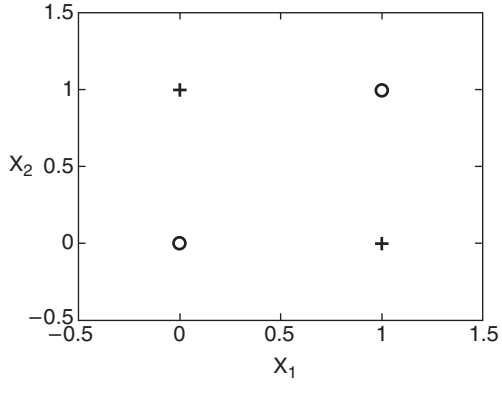



**Figure 6.22.** XOR classification problem. No linear hyperplane can separate the two classes.

### 6.7.2 Multi-layer Neural Network

A multi-layer neural network generalizes the basic concept of a perceptron to more complex architectures of nodes that are capable of learning nonlinear decision boundaries. A generic architecture of a multi-layer neural network is shown in Figure 6.23 where the nodes are arranged in groups called layers. These layers are commonly organized in the form of a chain such that every layer operates on the outputs of its preceding layer. In this way, the layers represent different levels of *abstraction* that are applied on the input features in a sequential manner. The composition of these abstractions generates the final output at the last layer, which is used for making predictions. In the following, we briefly describe the three types of layers used in multi-layer neural networks.

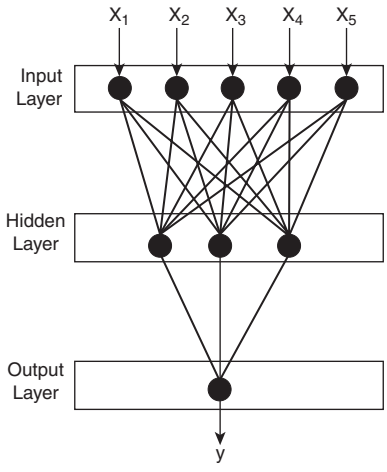
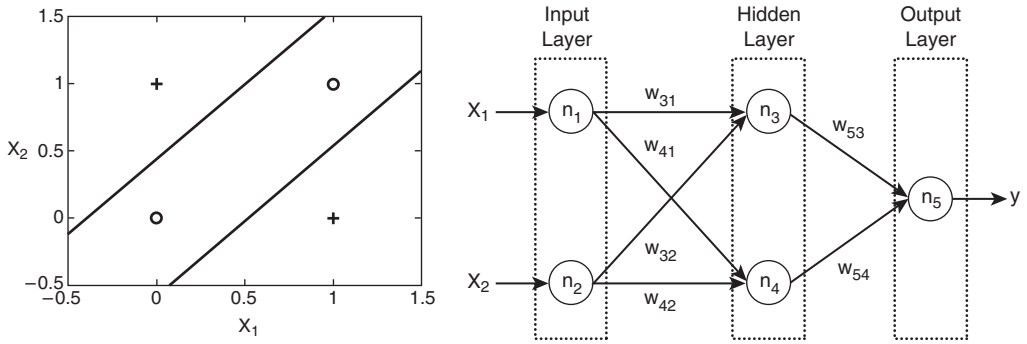



**Figure 6.23.** Example of a multi-layer artificial neural network (ANN).

The first layer of the network, called the **input layer**, is used for representing inputs from attributes. Every numerical or binary attribute is typically represented using a single node on this layer, while a categorical attribute is either represented using a different node for each categorical value, or by encoding the $k$-ary attribute using $\lceil \log_2 k \rceil$ input nodes. These inputs are fed into intermediary layers known as **hidden layers**, which are made up of processing units known as hidden nodes. Every hidden node operates on signals received from the input nodes or hidden nodes at the preceding layer, and produces an activation value that is transmitted to the next layer. The




(a) Decision boundary. (b) Neural network topology.

**Figure 6.24.** A two-layer neural network for the XOR problem.

final layer is called the **output layer** and processes the activation values from its preceding layer to produce predictions of output variables. For binary classification, the output layer contains a single node representing the binary class label. In this architecture, since the signals are propagated only in the forward direction from the input layer to the output layer, they are also called **feedforward neural networks**.

A major difference between multi-layer neural networks and perceptrons is the inclusion of hidden layers, which dramatically improves their ability to represent arbitrarily complex decision boundaries. For example, consider the XOR problem described in the previous section. The instances can be classified using two hyperplanes that partition the input space into their respective classes, as shown in Figure 6.24(a). Because a perceptron can create only one hyperplane, it cannot find the optimal solution. However, this problem can be addressed by using a hidden layer consisting of two nodes, as shown in Figure 6.24(b). Intuitively, we can think of each hidden node as a perceptron that tries to construct one of the two hyperplanes, while the output node simply combines the results of the perceptrons to yield the decision boundary shown in Figure 6.24(a).

The hidden nodes can be viewed as learning latent representations or *features* that are useful for distinguishing between the classes. While the first hidden layer directly operates on the input attributes and thus captures simpler features, the subsequent hidden layers are able to combine them and construct more complex features. From this perspective, multi-layer neural networks learn a hierarchy of features at different levels of abstraction that

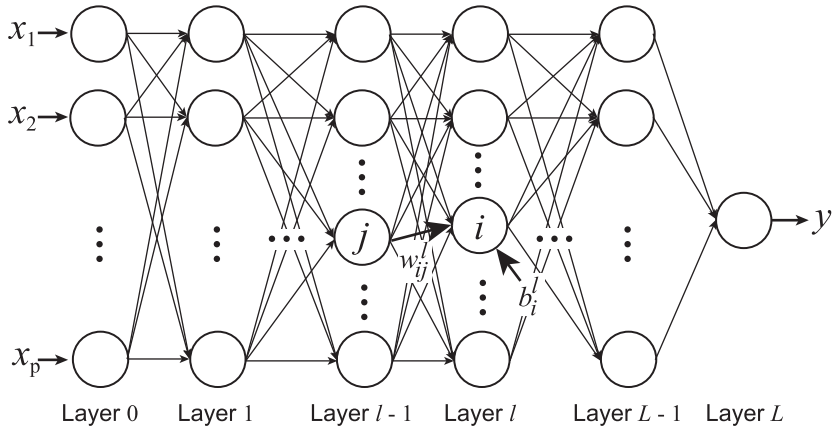



**Figure 6.25.** Schematic illustration of the parameters of an ANN model with $(L-1)$ hidden layers.

are finally combined at the output nodes to make predictions. Further, there are combinatorially many ways we can combine the features learned at the hidden layers of ANN, making them highly expressive. This property chiefly distinguishes ANN from other classification models such as decision trees, which can learn partitions in the attribute space but are unable to combine them in exponential ways.

To understand the nature of computations happening at the hidden and output nodes of ANN, consider the $i^{\text{th}}$ node at the $l^{\text{th}}$ layer of the network ($l > 0$), where the layers are numbered from 0 (input layer) to $L$ (output layer), as shown in Figure 6.25. The activation value generated at this node, $a_i^l$, can be represented as a function of the inputs received from nodes at the preceding layer. Let $w_{ij}^l$ represent the weight of the connection from the $j^{\text{th}}$ node at layer $(l-1)$ to the $i^{\text{th}}$ node at layer $l$. Similarly, let us denote the bias term at this node as $b_i^l$. The activation value $a_i^l$ can then be expressed as

$$a_i^l = f(z_i^l) = f\big(\sum_j w_{ij}^l a_j^{l-1} + b_i^l\big),$$

where $z$ is called the *linear predictor* and $f(.)$ is the activation function that converts $z$ to $a$. Further, note that, by definition, $a_j^0 = x_j$ at the input layer and $a^L = \hat{y}$ at the output node.

There are a number of alternate activation functions apart from the sign function that can be used in multi-layer neural networks. Some examples include linear, sigmoid (logistic), and hyperbolic tangent functions, as shown in Figure 6.26. These functions are able to produce real-valued and nonlinear

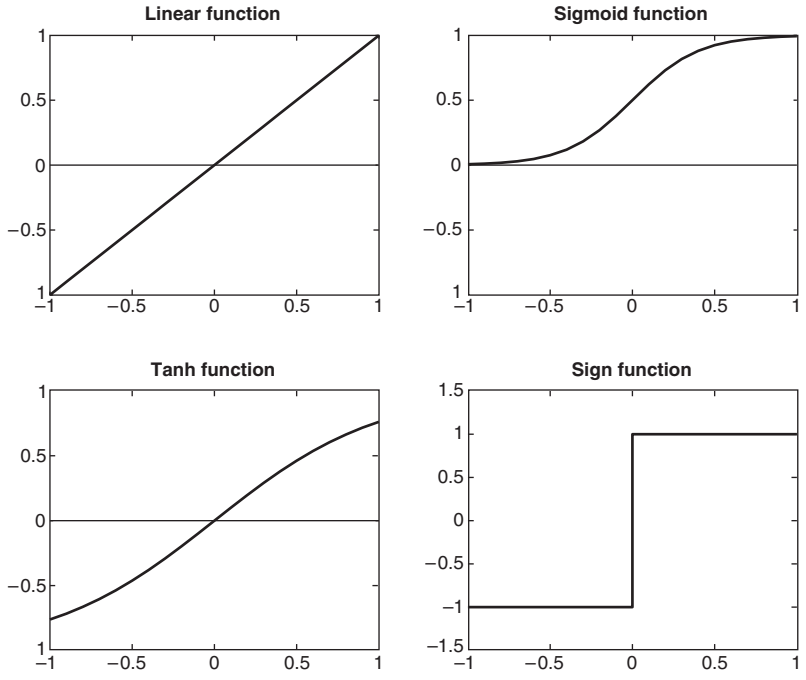



**Figure 6.26.** Types of activation functions used in multi-layer neural networks.

activation values. Among these activation functions, the sigmoid $\sigma(.)$ has been widely used in many ANN models, although the use of other types of activation functions in the context of deep learning will be discussed in Section 6.8. We can thus represent $a_i^l$ as

$$a_i^l = \sigma(z_i^l) = \frac{1}{1 + e^{-z_i^l}}. \tag{6.50}$$

### Learning Model Parameters

The weights and bias terms $(\mathbf{w}, \mathbf{b})$ of the ANN model are learned during training so that the predictions on training instances match the true labels. This is achieved by using a loss function

$$E(\mathbf{w}, \mathbf{b}) = \sum_{k=1}^{n} \text{Loss } (y_k,\ \hat{y}_k) \tag{6.51}$$

where $y_k$ is the true label of the $k^{\text{th}}$ training instance and $\hat{y}_k$ is equal to $a^L$, produced by using $\mathbf{x}_k$. A typical choice of the loss function is the **squared loss function**:.

$$\text{Loss } (y_k,\ \hat{y}_k) = (y_k - \hat{y}_k)^2. \tag{6.52}$$

Note that $E(\mathbf{w}, \mathbf{b})$ is a function of the model parameters $(\mathbf{w}, \mathbf{b})$ because the output activation value $a^L$ depends on the weights and bias terms. We are interested in choosing $(\mathbf{w}, \mathbf{b})$ that minimizes the training loss $E(\mathbf{w}, \mathbf{b})$. Unfortunately, because of the use of hidden nodes with nonlinear activation functions, $E(\mathbf{w}, \mathbf{b})$ is not a convex function of $\mathbf{w}$ and $\mathbf{b}$, which means that $E(\mathbf{w}, \mathbf{b})$ can have local minima that are not globally optimal. However, we can still apply standard optimization techniques such as the **gradient descent method** to arrive at a locally optimal solution. In particular, the weight parameter $w_{ij}^l$ and the bias term $b_i^l$ can be iteratively updated using the following equations:

$$w_{ij}^l \longleftarrow w_{ij}^l - \lambda \frac{\partial E}{\partial w_{ij}^l}, \tag{6.53}$$

$$b_i^l \longleftarrow b_i^l - \lambda \frac{\partial E}{\partial b_i^l}, \tag{6.54}$$

where $\lambda$ is a hyper-parameter known as the learning rate. The intuition behind this equation is to move the weights in a direction that reduces the training loss. If we arrive at a minima using this procedure, the gradient of the training loss will be close to 0, eliminating the second term and resulting in the convergence of weights. The weights are commonly initialized with values drawn randomly from a Gaussian or a uniform distribution.

A necessary tool for updating weights in Equation 6.53 is to compute the partial derivative of $E$ with respect to $w_{ij}^l$. This computation is non-trivial especially at hidden layers ($l < L$), since $w_{ij}^l$ does not directly affect $\hat{y} = aL$ (and hence the training loss), but has complex chains of influences via activation values at subsequent layers. To address this problem, a technique known as **backpropagation** was developed, which propagates the derivatives backward from the output layer to the hidden layers. This technique can be described as follows.

Recall that the training loss $E$ is simply the sum of individual losses at training instances. Hence the partial derivative of $E$ can be decomposed as a sum of partial derivatives of individual losses.

$$\frac{\partial E}{\partial w_j^l} = \sum_{k=1}^{n} \frac{\partial \text{ Loss } (y_k,\ \hat{y}_k)}{\partial w_j^l}.$$

To simplify discussions, we will consider only the derivatives of the loss at the $k^{\text{th}}$ training instance, which will be generically represented as $\text{Loss}(y, a^L)$. By using the chain rule of differentiation, we can represent the partial derivatives of the loss with respect to $w_{ij}^l$ as

$$\frac{\partial \text{ Loss}}{\partial w_{ij}^l} = \frac{\partial \text{ Loss}}{\partial a_i^l} \times \frac{\partial a_i^l}{\partial z_i^l} \times \frac{\partial z_i^l}{\partial w_{ij}^l}. \tag{6.55}$$

The last term of the previous equation can be written as

$$\frac{\partial z_i^l}{\partial w_{ij}^l} = \frac{\partial \left( \sum_j w_{ij}^l a_j^{l-1} + b_i^l \right)}{\partial w_{ij}^l} = a_j^{l-1}.$$

Also, if we use the sigmoid activation function, then

$$\frac{\partial a_i^l}{\partial z_i^l} = \frac{\partial \ \sigma(z_i^l)}{\partial z_i^l} = a_i^l(1 - a_i^l).$$

Equation 6.55 can thus be simplified as

$$\begin{aligned} \frac{\partial \text{ Loss}}{\partial w_{ij}^l} &= \delta_i^l \times a_i^l(1 - a_i^l) \times a_j^{l-1}, \\ \text{where } \delta_i^l &= \frac{\partial \text{ Loss}}{\partial a_i^l}. \end{aligned} \tag{6.56}$$

A similar formula for the partial derivatives with respect to the bias terms $b_i^l$ is given by

$$\frac{\partial \text{ Loss}}{\partial b_i^l} = \delta_i^l \times a_i^l(1 - a_i^l). \tag{6.57}$$

Hence, to compute the partial derivatives, we only need to determine $\delta_i^l$. Using a squared loss function, we can easily write $\delta^L$ at the output node as

$$\delta^L = \frac{\partial \text{ Loss}}{\partial a^L} = \frac{\partial \ (y - a^L)^2}{\partial a^L} = 2(a^L - y). \tag{6.58}$$

However, the approach for computing $\delta_j^l$ at hidden nodes ($l < L$) is more involved. Notice that $a_j^l$ affects the activation values $a_i^{l+1}$ of all nodes at the

next layer, which in turn influences the loss. Hence, again using the chain rule of differentiation, $\delta_j^l$ can be represented as

$$\begin{aligned}\delta_j^l &= \frac{\partial \text{ Loss}}{\partial a_j^l} = \sum_i \left(\frac{\partial \text{ Loss}}{\partial a_i^{l+1}} \times \frac{\partial a_i^{l+1}}{\partial a_j^l}\right). \\ &= \sum_i \left(\frac{\partial \text{ Loss}}{\partial a_i^{l+1}} \times \frac{\partial a_i^{l+1}}{\partial z_i^{l+1}} \times \frac{\partial z_i^{l+1}}{\partial a_j^l}\right). \\ &= \sum_i (\delta_i^{l+1} \times a_i^{l+1}(1 - a_i^{l+1}) \times w_{ij}^{l+1}). \end{aligned} \tag{6.59}$$

The previous equation provides a concise representation of the $\delta_j^l$ values at layer $l$ in terms of the $\delta_i^{l+1}$ values computed at layer $l+1$. Hence, proceeding backward from the output layer $L$ to the hidden layers, we can recursively apply Equation 6.59 to compute $\delta_i^l$ at every hidden node. $\delta_i^l$ can then be used in Equations 6.56 and 6.57 to compute the partial derivatives of the loss with respect to $w_{ij}^l$ and $b_i^l$, respectively. Algorithm 6.4 summarizes the complete approach for learning the model parameters of ANN using backpropagation and gradient descent method.

**Algorithm 6.4** Learning ANN using backpropagation and gradient descent.

1: Let $D.train = \{(\mathbf{x}_k, y_k) \mid k = 1, 2, \ldots, n\}$ be the set of training instances.
2: Set counter $c \leftarrow 0$.
3: Initialize the weight and bias terms $(\mathbf{w}^{(0)}, \mathbf{b}^{(0)})$ with random values.
4: **repeat**
5: **for** each training instance $(\mathbf{x}_k, y_k) \in D.train$ **do**
6: Compute the set of activations $(a_i^l)_k$ by making a forward pass using $\mathbf{x}_k$.
7: Compute the set $(\delta_i^l)_k$ by backpropagation using Equations 6.58 and 6.59.
8: Compute $(\partial \text{ Loss}/\partial w_{ij}^l,\ \partial \text{ Loss}/\partial b_i^l)_k$ using Equations 6.56 and 6.57.
9: **end for**
10: Compute $\partial E/\partial w_{ij}^l \longleftarrow \sum_{k=1}^n (\partial \text{ Loss}/\partial w_{ij}^l)_k$.
11: Compute $\partial E/\partial b_i^l \longleftarrow \sum_{k=1}^n (\partial \text{ Loss}/\partial b_i^l)_k$.
12: Update $(\mathbf{w}^{(c+1)}, \mathbf{b}^{(c+1)})$ by gradient descent using Equations 6.53 and 6.54.
13: Update $c \leftarrow c + 1$.
14: **until** $(\mathbf{w}^{(c+1)}, \mathbf{b}^{(c+1)})$ and $(\mathbf{w}^{(c)}, \mathbf{b}^{(c)})$ converge to the same value

### 6.7.3 Characteristics of ANN

1. Multi-layer neural networks with at least one hidden layer are **universal approximators**; i.e., they can be used to approximate any target function. They are thus highly expressive and can be used to learn complex decision boundaries in diverse applications. ANN can also be used for multiclass classification and regression problems, by appropriately modifying the output layer. However, the high model complexity of classical ANN models makes it susceptible to overfitting, which can be overcome to some extent by using deep learning techniques discussed in Section 6.8.3.

2. ANN provides a natural way to represent a hierarchy of features at multiple levels of abstraction. The outputs at the final hidden layer of the ANN model thus represent features at the highest level of abstraction that are most useful for classification. These features can also be used as inputs in other supervised classification models, e.g., by replacing the output node of the ANN by any generic classifier.

3. ANN represents complex high-level features as compositions of simpler lower-level features that are easier to learn. This provides ANN the ability to gradually increase the complexity of representations, by adding more hidden layers to the architecture. Further, since simpler features can be combined in combinatorial ways, the number of complex features learned by ANN is much larger than traditional classification models. This is one of the main reasons behind the high expressive power of deep neural networks.

4. ANN can easily handle irrelevant attributes, by using zero weights for attributes that do not help in improving the training loss. Also, redundant attributes receive similar weights and do not degrade the quality of the classifier. However, if the number of irrelevant or redundant attributes is large, the learning of the ANN model may suffer from overfitting, leading to poor generalization performance.

5. Since the learning of ANN model involves minimizing a non-convex function, the solutions obtained by gradient descent are not guaranteed to be globally optimal. For this reason, ANN has a tendency to get stuck in local minima, a challenge that can be addressed by using deep learning techniques discussed in Section 6.8.4.

6. Training an ANN is a time consuming process, especially when the number of hidden nodes is large. Nevertheless, test examples can be classified rapidly.

7. Just like logistic regression, ANN can learn in the presence of interacting variables, since the model parameters are jointly learned over all variables together. In addition, ANN cannot handle instances with missing values in the training or testing phase.

## 6.8 Deep Learning

As described above, the use of hidden layers in ANN is based on the general belief that complex high-level features can be constructed by combining simpler lower-level features. Typically, the greater the number of hidden layers, the deeper the hierarchy of features learned by the network. This motivates the learning of ANN models with long chains of hidden layers, known as **deep neural networks**. In contrast to "shallow" neural networks that involve only a small number of hidden layers, deep neural networks are able to represent features at multiple levels of abstraction and often require far fewer nodes per layer to achieve generalization performance similar to shallow networks.

Despite the huge potential in learning deep neural networks, it has remained challenging to learn ANN models with a large number of hidden layers using classical approaches. Apart from reasons related to limited computational resources and hardware architectures, there have been a number of algorithmic challenges in learning deep neural networks. First, learning a deep neural network with low training error has been a daunting task because of the saturation of sigmoid activation functions, resulting in slow convergence of gradient descent. This problem becomes even more serious as we move away from the output node to the hidden layers, because of the compounded effects of saturation at multiple layers, known as the **vanishing gradient problem**. Because of this reason, classical ANN models have suffered from slow and ineffective learning, leading to poor training and test performance. Second, the learning of deep neural networks is quite sensitive to the initial values of model parameters, chiefly because of the non-convex nature of the optimization function and the slow convergence of gradient descent. Third, deep neural networks with a large number of hidden layers have high model complexity, making them susceptible to overfitting. Hence, even if a deep neural network has been trained to show low training error, it can still suffer from poor generalization performance.

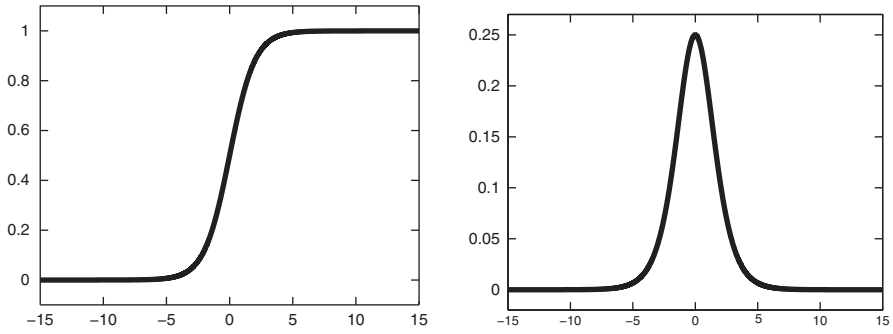
These challenges have deterred progress in building deep neural networks for several decades and it is only recently that we have started to unlock their immense potential with the help of a number of advances being made in the area of deep learning. Although some of these advances have been around for some time, they have only gained mainstream attention in the last decade, with deep neural networks continually beating records in various competitions and solving problems that were too difficult for other classification approaches.

There are two factors that have played a major role in the emergence of deep learning techniques. First, the availability of larger labeled data sets, e.g., the ImageNet data set contains more than 10 million labeled images, has made it possible to learn more complex ANN models than ever before, without falling easily into the traps of model overfitting. Second, advances in computational abilities and hardware infrastructures, such as the use of graphical processing units (GPU) for distributed computing, have greatly helped in experimenting with deep neural networks with larger architectures that would not have been feasible with traditional resources.

In addition to the previous two factors, there have been a number of algorithmic advancements to overcome the challenges faced by classical methods in learning deep neural networks. Some examples include the use of more responsive combinations of loss functions and activation functions, better initialization of model parameters, novel regularization techniques, more agile architecture designs, and better techniques for model learning and hyper-parameter selection. In the following, we describe some of the deep learning advances made to address the challenges in learning deep neural networks. Further details on recent developments in deep learning can be obtained from the Bibliographic Notes.

### 6.8.1 Using Synergistic Loss Functions

One of the major realizations leading to deep learning has been the importance of choosing appropriate combinations of activation and loss functions. Classical ANN models commonly made use of the sigmoid activation function at the output layer, because of its ability to produce real-valued outputs between 0 and 1, which was combined with a squared loss objective to perform gradient descent. It was soon noticed that this particular combination of activation and loss function resulted in the saturation of output activation values, which can be described as follows.




(a) $\sigma(z)$. (b) $\partial\sigma(z)/\partial z$.

**Figure 6.27.** Plots of sigmoid function and its derivative.

### Saturation of Outputs

Although the sigmoid has been widely-used as an activation function, it easily saturates at high and low values of inputs that are far away from 0. Observe from Figure 6.27(a) that $\sigma(z)$ shows variance in its values only when $z$ is close to 0. For this reason, $\partial\sigma(z)/\partial z$ is non-zero for only a small range of $z$ around 0, as shown in Figure 6.27(b). Since $\partial\sigma(z)/\partial z$ is one of the components in the gradient of loss (see Equation 6.55), we get a diminishing gradient value when the activation values are far from 0.

To illustrate the effect of saturation on the learning of model parameters at the output node, consider the partial derivative of loss with respect to the weight $w_j^L$ at the output node. Using the squared loss function, we can write this as

$$\frac{\partial \text{ Loss}}{\partial w_j^L} = 2(a^L - y) \times \sigma(z^L)(1 - \sigma(z^L)) \times a_j^{L-1}. \tag{6.60}$$

In the previous equation, notice that when $z^L$ is highly negative, $\sigma(z^L)$ (and hence the gradient) is close to 0. On the other hand, when $z^L$ is highly positive, $(1 - \sigma(z^L))$ becomes close to 0, nullifying the value of the gradient. Hence, irrespective of whether the prediction $a^L$ matches the true label $y$ or not, the gradient of the loss with respect to the weights is close to 0 whenever $z^L$ is highly positive or negative. This causes an unnecessarily slow convergence of the model parameters of the ANN model, often resulting in poor learning.

Note that it is the combination of the squared loss function and the sigmoid activation function at the output node that together results in diminishing gradients (and thus poor learning) upon saturation of outputs. It is thus

important to choose a synergistic combination of loss function and activation function that does not suffer from the saturation of outputs.

### Cross entropy loss function

The cross entropy loss function, which was described in the context of logistic regression in Section 6.6.2, can significantly avoid the problem of saturating outputs when used in combination with the sigmoid activation function. The cross entropy loss function of a real-valued prediction $\hat{y} \in (0, 1)$ on a data instance with binary label $y \in \{0, 1\}$ can be defined as

$$Loss\ (y,\ \hat{y}) = -y \log(\hat{y}) - (1 - y) \log(1 - \hat{y}), \tag{6.61}$$

where log represents the natural logarithm (to base $e$) and $0 \log(0) = 0$ for convenience. The cross entropy function has foundations in information theory and measures the amount of disagreement between $y$ and $\hat{y}$. The partial derivative of this loss function with respect to $\hat{y} = a^L$ can be given as

$$\begin{aligned} \delta^L = \frac{\partial\ Loss}{\partial a^L} &= \frac{-y}{a^L} + \frac{(1-y)}{(1-a^L)}. \\ &= \frac{(a^L - y)}{a^L(1-a^L)}. \end{aligned} \tag{6.62}$$

Using this value of $\delta^L$ in Equation 6.56, we can obtain the partial derivative of the loss with respect to the weight $w_j^l$ at the output node as

$$\begin{aligned} \frac{\partial\ \text{Loss}}{\partial w_j^L} &= \frac{(a^L - y)}{a^L(1-a^L)} \times a^L(1-a^L) \times a_j^{L-1}. \\ &= (a^L - y) \times a_j^{L-1}. \end{aligned} \tag{6.63}$$

Notice the simplicity of the previous formula using the cross entropy loss function. The partial derivatives of the loss with respect to the weights at the output node depend only on the difference between the prediction $a^L$ and the true label $y$. In contrast to Equation 6.60, it does not involve terms such as $\sigma(z^L)(1 - \sigma(z^L))$ that can be impacted by saturation of $z^L$. Hence, the gradients are high whenever $(a^L - y)$ is large, promoting effective learning of the model parameters at the output node. This has been a major breakthrough in the learning of modern ANN models and it is now a common practice to use the cross entropy loss function with sigmoid activations at the output node.

### 6.8.2 Using Responsive Activation Functions

Even though the cross entropy loss function helps in overcoming the problem of saturating outputs, it still does not solve the problem of saturation at hidden layers, arising due to the use of sigmoid activation functions at hidden nodes. In fact, the effect of saturation on the learning of model parameters is even more aggravated at hidden layers, a problem known as the vanishing gradient problem. In the following, we describe the vanishing gradient problem and the use of a more responsive activation function, called the **rectified linear output unit (ReLU)**, to overcome this problem.

#### Vanishing Gradient Problem

The impact of saturating activation values on the learning of model parameters increases at deeper hidden layers that are farther away from the output node. Even if the activation in the output layer does not saturate, the repeated multiplications performed as we backpropagate the gradients from the output layer to the hidden layers may lead to decreasing gradients in the hidden layers. This is called the vanishing gradient problem, which has been one of the major hindrances in learning deep neural networks.

To illustrate the vanishing gradient problem, consider an ANN model that consists of a single node at every hidden layer of the network, as shown in Figure 6.28. This simplified architecture involves a single chain of hidden nodes where a single weighted link $w^l$ connects the node at layer $l-1$ to the node at layer $l$. Using Equations 6.56 and 6.59, we can represent the partial derivative of the loss with respect to $w^l$ as

$$\begin{aligned} \frac{\partial \text{ Loss}}{\partial w^l} &= \delta^l \times a^l(1-a^l) \times a^{l-1}, \\ \text{where } \delta^l &= 2(a^L - y) \times \prod_{r=l}^{L-1} (a^{r+1}(1-a^{r+1}) \times w^{r+1}). \end{aligned} \tag{6.64}$$

Notice that if any of the linear predictors $z^{r+1}$ saturates at subsequent layers, then the term $a^{r+1}(1-a^{r+1})$ becomes close to 0, thus diminishing the overall gradient. The saturation of activations thus gets compounded and has

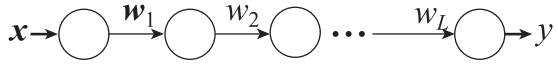



**Figure 6.28.** An example of an ANN model with only one node at every hidden layer.

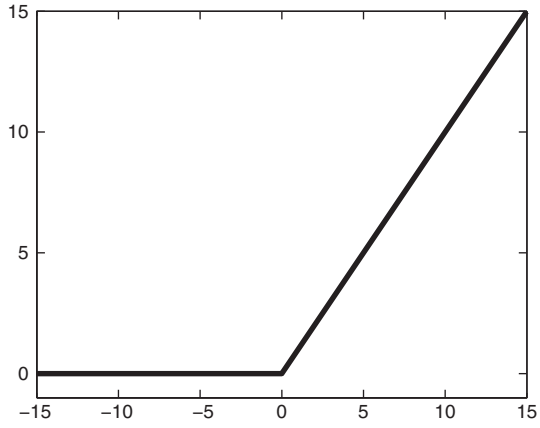



**Figure 6.29.** Plot of the rectified linear unit (ReLU) activation function.

multiplicative effects on the gradients at hidden layers, making them highly unstable and thus, unsuitable for use with gradient descent. Even though the previous discussion only pertains to the simplified architecture involving a single chain of hidden nodes, a similar argument can be made for any generic ANN architecture involving multiple chains of hidden nodes. Note that the vanishing gradient problem primarily arises because of the use of sigmoid activation function at hidden nodes, which is known to easily saturate especially after repeated multiplications.

### Rectified Linear Units (ReLU)

To overcome the vanishing gradient problem, it is important to use an activation function $f(z)$ at the hidden nodes that provides a stable and significant value of the gradient whenever a hidden node is active, i.e., $z > 0$. This is achieved by using rectified linear units (ReLU) as activation functions at hidden nodes, which can be defined as

$$a = f(z) \quad = \quad \begin{cases} z, & \text{if } z > 0. \\ 0, & \text{otherwise.} \end{cases} \tag{6.65}$$

The idea of ReLU has been inspired from biological neurons, which are either in an inactive state ($f(z) = 0$) or show an activation value proportional to the input. Figure 6.29 shows a plot of the ReLU function. We can see that it is linear with respect to $z$ when $z > 0$. Hence, the gradient of the activation

value with respect to $z$ can be written as

$$\frac{\partial a}{\partial z} = \begin{cases} 1, & \text{if } z > 0. \\ 0, & \text{if } z < 0. \end{cases} \tag{6.66}$$

Although $f(z)$ is not differentiable at 0, it is common practice to use $\partial a/\partial z = 0$ when $z = 0$. Since the gradient of the ReLU activation function is equal to 1 whenever $z > 0$, it avoids the problem of saturation at hidden nodes, even after repeated multiplications. Using ReLU, the partial derivatives of the loss with respect to the weight and bias parameters can be given by

$$\frac{\partial \text{ Loss}}{\partial w_{ij}^l} = \delta_i^l \times I(z_i^l) \times a_j^{l-1}, \tag{6.67}$$

$$\frac{\partial \text{ Loss}}{\partial b_i^l} = \delta_i^l \times I(z_i^l), \tag{6.68}$$

$$\text{where } \delta_i^l = \sum_{i=1}^{n} (\delta_i^{l+1} \times I(z_i^{l+1}) \times w_{ij}^{l+1}),$$

$$\text{and } I(z) = \begin{cases} 1, & \text{if } z > 0. \\ 0, & \text{otherwise.} \end{cases}$$

Notice that ReLU shows a linear behavior in the activation values whenever a node is active, as compared to the nonlinear properties of the sigmoid function. This linearity promotes better flows of gradients during backpropagation, and thus simplifies the learning of ANN model parameters. The ReLU is also highly responsive at large values of $z$ away from 0, as opposed to the sigmoid activation function, making it more suitable for gradient descent. These differences give ReLU a major advantage over the sigmoid function. Indeed, ReLU is used as the preferred choice of activation function at hidden layers in most modern ANN models.

### 6.8.3 Regularization

A major challenge in learning deep neural networks is the high model complexity of ANN models, which grows with the addition of hidden layers in the network. This can become a serious concern, especially when the training set is small, due to the phenomena of model overfitting. To overcome this challenge, it is important to use techniques that can help in reducing

the complexity of the learned model, known as **regularization techniques**. Classical approaches for learning ANN models did not have an effective way to promote regularization of the learned model parameters. Hence, they had often been sidelined by other classification methods, such as support vector machines (SVM), which have in-built regularization mechanisms. (SVMs will be discussed in more detail in Section 6.9).

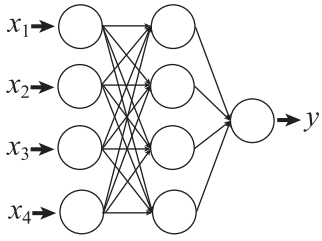
One of the major advancements in deep learning has been the development of novel regularization techniques for ANN models that are able to offer significant improvements in generalization performance. In the following, we discuss one of the regularization techniques for ANN, known as the **dropout** method, that have gained a lot of attention in several applications.

### Dropout

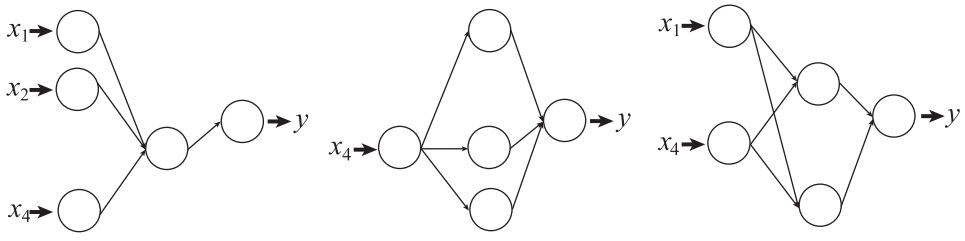
The main objective of dropout is to avoid the learning of spurious features at hidden nodes, occurring due to model overfitting. It uses the basic intuition that spurious features often "co-adapt" themselves such that they show good training performance only when used in highly selective combinations. On the other hand, relevant features can be used in a diversity of feature combinations and hence are quite resilient to the removal or modification of other features. The dropout method uses this intuition to break complex "co-adaptations" in the learned features by randomly dropping input and hidden nodes in the network during training.

Dropout belongs to a family of regularization techniques that uses the criteria of resilience to random perturbations as a measure of the robustness (and hence, simplicity) of a model. For example, one approach to regularization is to inject noise in the input attributes of the training set and learn a model with the noisy training instances. If a feature learned from the training data is indeed generalizable, it should not be affected by the addition of noise. Dropout can be viewed as a similar regularization approach that perturbs the information content of the training set not only at the level of attributes but also at multiple levels of abstractions, by dropping input and hidden nodes.

The dropout method draws inspiration from the biological process of gene swapping in sexual reproduction, where half of the genes from both parents are combined together to create the genes of the offspring. This favors the selection of parent genes that are not only useful but can also inter-mingle with diverse combinations of genes coming from the other parent. On the other hand, co-adapted genes that function only in highly selective combinations are soon eliminated in the process of evolution. This idea is used in the dropout




(a) Original network.




(b) Sub-networks.

**Figure 6.30.** Examples of sub-networks generated in the dropout method using $\gamma = 0.5$.

method for eliminating spurious co-adapted features. A simplified description of the dropout method is provided in the rest of this section.

Let $(\mathbf{w}^k, \mathbf{b}^k)$ represent the model parameters of the ANN model at the $k^{\text{th}}$ iteration of the gradient descent method. At every iteration, we randomly select a fraction $\gamma$ of input and hidden nodes to be dropped from the network, where $\gamma \in (0, 1)$ is a hyper-parameter that is typically chosen to be 0.5. The weighted links and bias terms involving the dropped nodes are then eliminated, resulting in a "thinned" sub-network of smaller size. The model parameters of the sub-network $(\mathbf{w}_s^k, \mathbf{b}_s^k)$ are then updated by computing activation values and performing backpropagation on this smaller sub-network. These updated values are then added back in the original network to obtain the updated model parameters, $(\mathbf{w}^{k+1}, \mathbf{b}^{k+1})$, to be used in the next iteration.

Figure 6.30 shows some examples of sub-networks that can be generated at different iterations of the dropout method, by randomly dropping input and hidden nodes. Since every sub-network has a different architecture, it is difficult to learn complex co-adaptations in the features that can result in overfitting. Instead, the features at the hidden nodes are learned to be more

agile to random modifications in the network structure, thus improving their generalization ability. The model parameters are updated using a different random sub-network at every iteration, till the gradient descent method converges.

Let $(\mathbf{w}^{k_{max}}, \mathbf{b}^{k_{max}})$ denote the model parameters at the last iteration $k_{max}$ of the gradient descent method. These parameters are finally scaled down by a factor of $(1 - \gamma)$, to produce the weights and bias terms of the final ANN model, as follows:

$$(\mathbf{w}^*, \mathbf{b}^*) = ((1 - \gamma) \times \mathbf{w}^{k_{max}}, (1 - \gamma) \times \mathbf{b}^{k_{max}})$$

We can now use the complete neural network with model parameters $(\mathbf{w}^*, \mathbf{b}^*)$ for testing. The dropout method has been shown to provide significant improvements in the generalization performance of ANN models in a number of applications. It is computationally cheap and can be applied in combination with any of the other deep learning techniques. It also has a number of similarities with a widely-used ensemble learning method known as **bagging**, which learns multiple models using random subsets of the training set, and then uses the *average output* of all the models to make predictions. (Bagging will be presented in more detail later in Section 6.10.4). In a similar vein, it can be shown that the predictions of the final network learned using dropout approximates the average output of all possible $2^n$ sub-networks that can be formed using $n$ nodes. This is one of the reasons behind the superior regularization abilities of dropout.

### 6.8.4 Initialization of Model Parameters

Because of the non-convex nature of the loss function used by ANN models, it is possible to get stuck in locally optimal but globally inferior solutions. Hence, the initial choice of model parameter values plays a significant role in the learning of ANN by gradient descent. The impact of poor initialization is even more aggravated when the model is complex, the network architecture is deep, or the classification task is difficult. In such cases, it is often advisable to first learn a simpler model for the problem, e.g., using a single hidden layer, and then incrementally increase the complexity of the model, e.g., by adding more hidden layers. An alternate approach is to train the model for a simpler task and then use the learned model parameters as initial parameter choices in the learning of the original task. The process of initializing ANN model parameters before the actual training process is known as **pretraining**.

Pretraining helps in initializing the model to a suitable region in the parameter space that would otherwise be inaccessible by random initialization.

Pretraining also reduces the variance in the model parameters by fixing the starting point of gradient descent, thus reducing the chances of overfitting due to multiple comparisons. The models learned by pretraining are thus more consistent and provide better generalization performance.

### Supervised Pretraining

A common approach for pretraining is to incrementally train the ANN model in a layer-wise manner, by adding one hidden layer at a time. This approach, known as **supervised pretraining**, ensures that the parameters learned at every layer are obtained by solving a simpler problem, rather than learning all model parameters together. These parameter values thus provide a good choice for initializing the ANN model. The approach for supervised pretraining can be briefly described as follows.

We start the supervised pretraining process by considering a reduced ANN model with only a single hidden layer. By applying gradient descent on this simple model, we are able to learn the model parameters of the first hidden layer. At the next run, we add another hidden layer to the model and apply gradient descent to learn the parameters of the newly added hidden layer, while keeping the parameters of the first layer fixed. This procedure is recursively applied such that while learning the parameters of the $l^{\text{th}}$ hidden layer, we consider a reduced model with only $l$ hidden layers, whose first $(l-1)$ hidden layers are not updated on the $l^{\text{th}}$ run but are instead fixed using pretrained values from previous runs. In this way, we are able to learn the model parameters of all $(L-1)$ hidden layers. These pretrained values are used to initialize the hidden layers of the final ANN model, which is fine-tuned by applying a final round of gradient descent over all the layers.

### Unsupervised Pretraining

Supervised pretraining provides a powerful way to initialize model parameters, by gradually growing the model complexity from shallower to deeper networks. However, supervised pretraining requires a sufficient number of labeled training instances for effective initialization of the ANN model. An alternate pretraining approach is **unsupervised pretraining**, which initializes model parameters by using unlabeled instances that are often abundantly available. The basic idea of unsupervised pretraining is to initialize the ANN model in such a way that the learned features capture the latent structure in the unlabeled data.

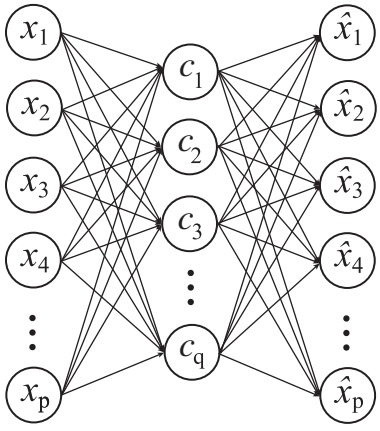



**Figure 6.31.** The basic architecture of a single-layer autoencoder.

Unsupervised pretraining relies on the assumption that learning the distribution of the input data can indirectly help in learning the classification model. It is most helpful when the number of labeled examples is small and the features for the supervised problem bear resemblance to the factors generating the input data. Unsupervised pretraining can be viewed as a different form of regularization, where the focus is not explicitly toward finding simpler features but instead toward finding features that can best explain the input data. Historically, unsupervised pretraining has played an important role in reviving the area of deep learning, by making it possible to train any generic deep neural network without requiring specialized architectures.

### Use of Autoencoders

One simple and commonly used approach for unsupervised pretraining is to use an unsupervised ANN model known as an **autoencoder**. The basic architecture of an autoencoder is shown in Figure 6.31. An autoencoder attempts to learn a *reconstruction* of the input data by mapping the attributes $\mathbf{x}$ to latent features $\mathbf{c}$, and then re-projecting $\mathbf{c}$ back to the original attribute space to create the reconstruction $\hat{\mathbf{x}}$. The latent features are represented using a hidden layer of nodes, while the input and output layers represent the attributes and contain the same number of nodes. During training, the goal is to learn an autoencoder model that provides the lowest reconstruction error, $RE(\mathbf{x}, \hat{\mathbf{x}})$, on all input data instances. A typical choice of the reconstruction error is the squared loss function:

$$RE(\mathbf{x}, \hat{\mathbf{x}}) = ||\mathbf{x} - \hat{\mathbf{x}}||^2.$$

The model parameters of the autoencoder can be learned by using a similar gradient descent method as the one used for learning supervised ANN models for classification. The key difference is the use of the reconstruction error on all training instances as the training loss. Autoencoders that have multiple layers of hidden layers are known as **stacked autoencoders**.

Autoencoders are able to capture complex representations of the input data by the use of hidden nodes. However, if the number of hidden nodes is large, it is possible for an autoencoder to learn the identity relationship, where the input $\mathbf{x}$ is just copied and returned as the output $\hat{\mathbf{x}}$, resulting in a trivial solution. For example, if we use as many hidden nodes as the number of attributes, then it is possible for every hidden node to copy an attribute and simply pass it along to an output node, without extracting any useful information. To avoid this problem, it is common practice to keep the number of hidden nodes smaller than the number of input attributes. This forces the autoencoder to learn a compact and useful *encoding* of the input data, similar to a dimensionality reduction technique. An alternate approach is to corrupt the input instances by adding random noise, and then learn the autoencoder to reconstruct the original instance from the noisy input. This approach is known as the **denoising autoencoder**, which offers strong regularization capabilities and is often used to learn complex features even in the presence of a large number of hidden nodes.

To use an autoencoder for unsupervised pretraining, we can follow a similar layer-wise approach like supervised pretraining. In particular, to pretrain the model parameters of the $l^{\text{th}}$ hidden layer, we can construct a reduced ANN model with only $l$ hidden layers and an output layer containing the same number of nodes as the attributes and is used for reconstruction. The parameters of the $l^{\text{th}}$ hidden layer of this network are then learned using a gradient descent method to minimize the reconstruction error. The use of unlabeled data can be viewed as providing *hints* to the learning of parameters at every layer that aid in generalization. The final model parameters of the ANN model are then learned by applying gradient descent over all the layers, using the initial values of parameters obtained from pretraining.

### Hybrid Pretraining

Unsupervised pretraining can also be combined with supervised pretraining by using two output layers at every run of pretraining, one for reconstruction and the other for supervised classification. The parameters of the $l^{\text{th}}$ hidden layer are then learned by jointly minimizing the losses on both output layers, usually weighted by a trade-off hyper-parameter $\alpha$. Such a combined approach often

shows better generalization performance than either of the approaches, since it provides a way to balance between the competing objectives of representing the input data and improving classification performance.

### 6.8.5 Characteristics of Deep Learning

Apart from the basic characteristics of ANN discussed in Section 6.7.3, the use of deep learning techniques provides the following additional characteristics:

1. An ANN model trained for some task can be easily re-used for a different task that involves the same attributes, by using pretraining strategies. For example, we can use the learned parameters of the original task as initial parameter choices for the target task. In this way, ANN promotes *re-usability* of learning, which can be quite useful when the target application has a smaller number of labeled training instances.

2. Deep learning techniques for regularization, such as the dropout method, help in reducing the model complexity of ANN and thus promoting good generalization performance. The use of regularization techniques is especially useful in high-dimensional settings, where the number of training labels is small but the classification problem is inherently difficult.

3. The use of an autoencoder for pretraining can help eliminate irrelevant attributes that are not related to other attributes. Further, it can help reduce the impact of redundant attributes by representing them as copies of the same attribute.

4. Although the learning of an ANN model can succumb to finding inferior and locally optimal solutions, there are a number of deep learning techniques that have been proposed to ensure adequate learning of an ANN. Apart from the methods discussed in this section, some other techniques involve novel architecture designs such as skip connections between the output layer and lower layers, which aids the easy flow of gradients during backpropagation.

5. A number of specialized ANN architectures have been designed to handle a variety of input data sets. Some examples include **convolutional neural networks** (CNN) for two-dimensional gridded objects such as images, and **recurrent neural network** (RNN) for sequences. While CNNs have been extensively used in the area of computer vision, RNNs have found applications in processing speech and language.

## 6.9 Support Vector Machine (SVM)

A support vector machine (SVM) is a discriminative classification model that learns linear or nonlinear decision boundaries in the attribute space to separate the classes. Apart from maximizing the separability of the two classes, SVM offers strong regularization capabilities, i.e., it is able to control the complexity of the model in order to ensure good generalization performance. Due to its unique ability to innately regularize its learning, SVM is able to learn highly expressive models without suffering from overfitting. It has thus received considerable attention in the machine learning community and is commonly used in several practical applications, ranging from handwritten digit recognition to text categorization. SVM has strong roots in statistical learning theory and is based on the principle of structural risk minimization. Another unique aspect of SVM is that it represents the decision boundary using only a subset of the training examples that are most difficult to classify, known as the **support vectors**. Hence, it is a discriminative model that is impacted only by training instances near the boundary of the two classes, in contrast to learning the generative distribution of every class.

To illustrate the basic idea behind SVM, we first introduce the concept of the margin of a separating hyperplane and the rationale for choosing such a hyperplane with maximum margin. We then describe how a linear SVM can be trained to explicitly look for this type of hyperplane. We conclude by showing how the SVM methodology can be extended to learn nonlinear decision boundaries by using kernel functions.

### 6.9.1 Margin of a Separating Hyperplane

The generic equation of a separating hyperplane can be written as

$$\mathbf{w}^T\mathbf{x} + b = 0,$$

where $\mathbf{x}$ represents the attributes and ($\mathbf{w}$, $b$) represent the parameters of the hyperplane. A data instance $\mathbf{x}_i$ can belong to either side of the hyperplane depending on the sign of $(\mathbf{w}^T\mathbf{x}_i + b)$. For the purpose of binary classification, we are interested in finding a hyperplane that places instances of both classes on opposite sides of the hyperplane, thus resulting in a *separation* of the two classes. If there exists a hyperplane that can perfectly separate the classes in the data set, we say that the data set is **linearly separable**. Figure 6.32 shows an example of linearly separable data involving two classes, squares and circles. Note that there can be infinitely many hyperplanes that can separate

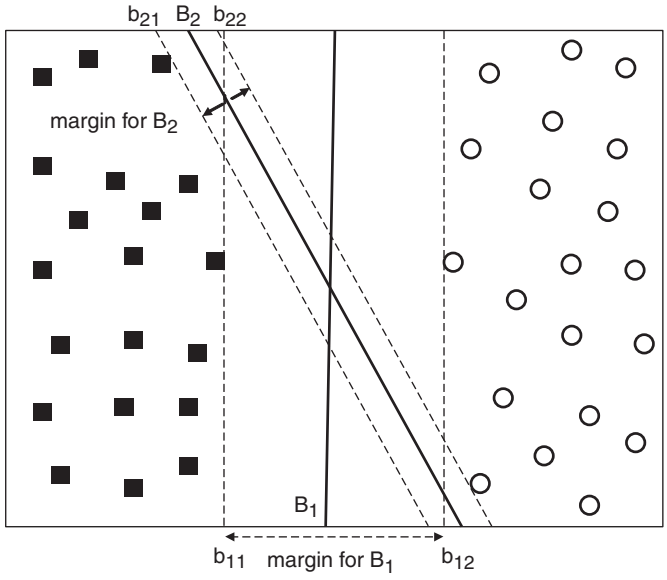



**Figure 6.32.** Margin of a hyperplane in a two-dimensional data set.

the classes, two of which are shown in Figure 6.32 as lines $B_1$ and $B_2$. Even though every such hyperplane will have zero training error, they can provide different results on previously unseen instances. Which separating hyperplane should we thus finally choose to obtain the best generalization performance? Ideally, we would like to choose a *simple* hyperplane that is robust to small perturbations. This can be achieved by using the concept of the margin of a separating hyperplane, which can be briefly described as follows.

For every separating hyperplane $B_i$, let us associate a pair of parallel hyperplanes, $b_{i1}$ and $b_{i2}$, such that they touch the closest instances of both classes, respectively. For example, if we move $B_1$ parallel to its direction, we can touch the first square using $b_{11}$ and the first circle using $b_{12}$. $b_{i1}$ and $b_{i2}$ are known as the **margin hyperplanes** of $B_i$ and the distance between them is known as the **margin** of the separating hyperplane $B_i$. From the diagram shown in Figure 6.32, notice that the margin for $B_1$ is considerably larger than that for $B_2$. In this example, $B_1$ turns out to be the separating hyperplane with the maximum margin, known as the **maximum margin hyperplane**.

### Rationale for Maximum Margin

Hyperplanes with large margins tend to have better generalization performance than those with small margins. Intuitively, if the margin is small, then

any slight perturbation in the hyperplane or the training instances located at the boundary can have quite an impact on the classification performance. Small margin hyperplanes are thus more susceptible to overfitting, as they are barely able to separate the classes with a very narrow room to allow perturbations. On the other hand, a hyperplane that is farther away from training instances of both classes has sufficient leeway to be robust to minor modifications in the data, and thus shows superior generalization performance.

The idea of choosing the maximum margin separating hyperplane also has strong foundations in statistical learning theory. It can be shown that the margin of such a hyperplane is inversely related to the $VC$-dimension of the classifier, which is a commonly used measure of the complexity of a model. As discussed in Section 3.4, a simpler model should be preferred over a more complex model if they both show similar training performance. Hence, maximizing the margin results in the selection of a separating hyperplane with the lowest model complexity, which is expected to show better generalization performance.

### 6.9.2 Linear SVM

A linear SVM is a classifier that searches for a separating hyperplane with the largest margin, which is why it is often known as a **maximal margin classifier**. The basic idea of SVM can be described as follows.

Consider a binary classification problem consisting of $n$ training instances, where every training instance $\mathbf{x}_i$ is associated with a binary label $y_i \in \{-1, 1\}$. Let $\mathbf{w}^T\mathbf{x} + b = 0$ be the equation of a separating hyperplane that separates the two classes by placing them on opposite sides. This means that

$$\begin{aligned} \mathbf{w}^T\mathbf{x}_i + b > 0 \quad & \text{if } y_i = 1, \\ \mathbf{w}^T\mathbf{x}_i + b < 0 \quad & \text{if } y_i = -1. \end{aligned}$$

The distance of any point $\mathbf{x}$ from the hyperplane is then given by

$$D(\mathbf{x}) = \frac{|\mathbf{w}^T\mathbf{x} + b|}{||\mathbf{w}||}$$

where $|.|$ denotes the absolute value and $||.||$ denotes the length of a vector. Let the distance of the closest point from the hyperplane with $y = 1$ be $k_+ > 0$. Similarly, let $k_- > 0$ denote the distance of the closest point from class $-1$.

This can be represented using the following constraints:

$$\begin{aligned} \frac{\mathbf{w}^T\mathbf{x}_i+b}{||\mathbf{w}||} &\geq k_+ \qquad \text{if } y_i = 1, \\ \frac{\mathbf{w}^T\mathbf{x}_i+b}{||\mathbf{w}||} &\leq -k_- \qquad \text{if } y_i = -1. \end{aligned} \tag{6.69}$$

The previous equations can be succinctly represented by using the product of $y_i$ and $(\mathbf{w}^T\mathbf{x}_i + b)$ as

$$y_i(\mathbf{w}^T\mathbf{x}_i + b) \geq M||\mathbf{w}|| \tag{6.70}$$

where $M$ is a parameter related to the margin of the hyperplane, i.e., if $k_+ = k_- = M$, then margin $= k_+ - k_- = 2M$. In order to find the maximum margin hyperplane that adheres to the previous constraints, we can consider the following optimization problem:

$$\begin{aligned} &\max_{\mathbf{w},b} \; M \\ \text{subject to} \quad &y_i(\mathbf{w}^T\mathbf{x}_i + b) \geq M||\mathbf{w}||. \end{aligned} \tag{6.71}$$

To find the solution to the previous problem, note that if $\mathbf{w}$ and $b$ satisfy the constraints of the previous problem, then any scaled version of $\mathbf{w}$ and $b$ would satisfy them too. Hence, we can conveniently choose $||\mathbf{w}|| = 1/M$ to simplify the right-hand side of the inequalities. Furthermore, maximizing $M$ amounts to minimizing $||\mathbf{w}||^2$. Hence, the optimization problem of SVM is commonly represented in the following form:

$$\begin{aligned} &\min_{\mathbf{w},b} \; \frac{||\mathbf{w}||^2}{2} \\ \text{subject to} \quad &y_i(\mathbf{w}^T\mathbf{x}_i + b) \geq 1. \end{aligned} \tag{6.72}$$

### Learning Model Parameters

Equation 6.72 represents a constrained optimization problem with linear inequalities. Since the objective function is convex and quadratic with respect to $\mathbf{w}$, it is known as a quadratic programming problem (QPP), which can be solved using standard optimization techniques, as described in Appendix E. In the following, we present a brief sketch of the main ideas for learning the model parameters of SVM.

First, we rewrite the objective function in a form that takes into account the constraints imposed on its solutions. The new objective function is known

as the **Lagrangian primal problem**, which can be represented as follows,

$$L_P = \frac{1}{2}\|\mathbf{w}\|^2 - \sum_{i=1}^{n} \lambda_i \Big( y_i(\mathbf{w}^T \mathbf{x}_i + b) - 1 \Big), \tag{6.73}$$

where the parameters $\lambda_i \geq 0$ correspond to the constraints and are called the **Lagrange multipliers**. Next, to minimize the Lagrangian, we take the derivative of $L_P$ with respect to $\mathbf{w}$ and $b$ and set them equal to zero:

$$\frac{\partial L_p}{\partial \mathbf{w}} = 0 \Longrightarrow \mathbf{w} = \sum_{i=1}^{n} \lambda_i y_i \mathbf{x}_i, \tag{6.74}$$

$$\frac{\partial L_p}{\partial b} = 0 \Longrightarrow \sum_{i=1}^{n} \lambda_i y_i = 0. \tag{6.75}$$

Note that using Equation 6.74, we can represent $\mathbf{w}$ completely in terms of the Lagrange multipliers. There is another relationship between $(\mathbf{w}, b)$ and $\lambda_i$ that is derived from the Karush-Kuhn-Tucker (KKT) conditions, a commonly used technique for solving QPP. This relationship can be described as

$$\lambda_i \big[ y_i(\mathbf{w}^T \mathbf{x}_i + b) - 1 \big] = 0. \tag{6.76}$$

Equation 6.76 is known as the **complementary slackness condition**, which sheds light on a valuable property of SVM. It states that the Lagrange multiplier $\lambda_i$ is strictly greater than 0 only when $\mathbf{x}_i$ satisfies the equation $y_i(\mathbf{w} \cdot \mathbf{x}_i + b) = 1$, which means that $\mathbf{x}_i$ lies exactly on a margin hyperplane. However, if $\mathbf{x}_i$ is farther away from the margin hyperplanes such that $y_i(\mathbf{w} \cdot \mathbf{x}_i + b) > 1$, then $\lambda_i$ is necessarily 0. Hence, $\lambda_i > 0$ for only a small number of instances that are closest to the separating hyperplane, which are known as **support vectors**. Figure 6.33 shows the support vectors of a hyperplane as filled circles and squares. Further, if we look at Equation 6.74, we will observe that training instances with $\lambda_i = 0$ do not contribute to the weight parameter $\mathbf{w}$. This suggests that $\mathbf{w}$ can be concisely represented only in terms of the support vectors in the training data, which are quite fewer than the overall number of training instances. This ability to represent the decision function only in terms of the support vectors is what gives this classifier the name support vector machines.

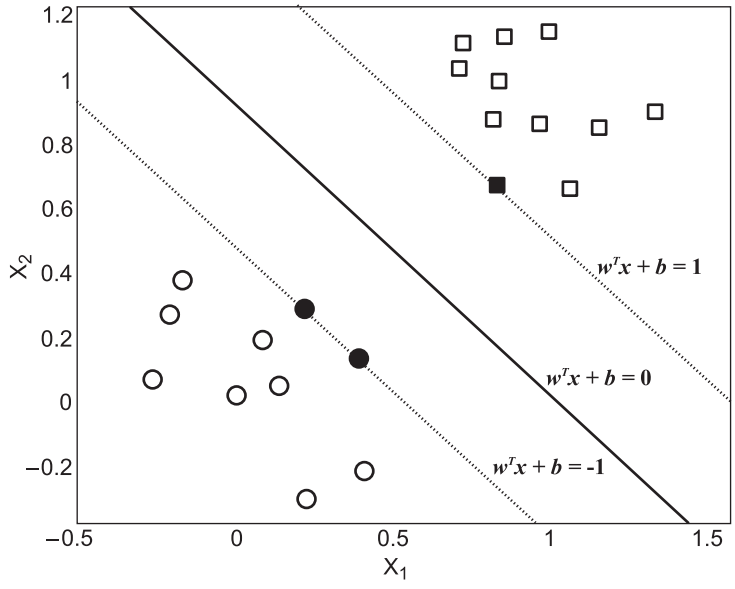



**Figure 6.33.** Support vectors of a hyperplane shown as filled circles and squares.

Using equations 6.74, 6.75, and 6.76 in Equation 6.73, we obtain the following optimization problem in terms of the Lagrange multipliers $\lambda_i$:

$$\max_{\lambda_i} \quad \sum_{i=1}^{n} \lambda_i - \frac{1}{2} \sum_{i=1}^{n} \sum_{j=1}^{n} \lambda_i \lambda_j y_i y_j \mathbf{x}_i^T \mathbf{x}_j \tag{6.77}$$

$$\text{subject to} \quad \sum_{i=1}^{n} \lambda_i y_i = 0,$$

$$\lambda_i \geq 0.$$

The previous optimization problem is called the **dual optimization problem**. Maximizing the dual problem with respect to $\lambda_i$ is equivalent to minimizing the primal problem with respect to $\mathbf{w}$ and $b$.

The key differences between the dual and primal problems are as follows:

1. Solving the dual problem helps us identify the support vectors in the data that have non-zero values of $\lambda_i$. Further, the solution of the dual problem is influenced only by the support vectors that are closest to the decision boundary of SVM. This helps in summarizing the learning of SVM solely in terms of its support vectors, which are easier to manage computationally. Further, it represents a unique ability of SVM to be

dependent only on the instances closest to the boundary, which are harder to classify, rather than the distribution of instances farther away from the boundary.

2. The objective of the dual problem involves only terms of the form $\mathbf{x}_i^T \mathbf{x}_j$, which are basically inner products in the attribute space. As we will see later in Section 6.9.4, this property will prove to be quite useful in learning nonlinear decision boundaries using SVM.

Because of these differences, it is useful to solve the dual optimization problem using any of the standard solvers for QPP. Having found an optimal solution for $\lambda_i$, we can use Equation 6.74 to solve for $\mathbf{w}$. We can then use Equation 6.76 on the support vectors to solve for $b$ as follows:

$$b = \frac{1}{n_S} \sum_{i \in S} \frac{1 - y_i \mathbf{w}^T \mathbf{x}_i}{y_i} \tag{6.78}$$

where $S$ represents the set of support vectors ($S = \{i | \lambda_i > 0\}$) and $n_S$ is the number of support vectors. The maximum margin hyperplane can then be expressed as

$$f(\mathbf{x}) = \left( \sum_{i=1}^{n} \lambda_i y_i \mathbf{x}_i^T \mathbf{x} \right) + b = 0. \tag{6.79}$$

Using this separating hyperplane, a test instance $\mathbf{x}$ can be assigned a class label using the sign of $f(\mathbf{x})$.

**Example 6.7.** Consider the two-dimensional data set shown in Figure 6.34, which contains eight training instances. Using quadratic programming, we can solve the optimization problem stated in Equation 6.77 to obtain the Lagrange multiplier $\lambda_i$ for each training instance. The Lagrange multipliers are depicted in the last column of the table. Notice that only the first two instances have non-zero Lagrange multipliers. These instances correspond to the support vectors for this data set.

Let $\mathbf{w} = (w_1, w_2)$ and $b$ denote the parameters of the decision boundary. Using Equation 6.74, we can solve for $w_1$ and $w_2$ in the following way:

$$w_1 = \sum_i \lambda_i y_i x_{i1} = 65.5261 \times 1 \times 0.3858 + 65.5261 \times -1 \times 0.4871 = -6.64.$$

$$w_2 = \sum_i \lambda_i y_i x_{i2} = 65.5261 \times 1 \times 0.4687 + 65.5261 \times -1 \times 0.611 = -9.32.$$

| $x_1$ | $x_2$ | y | Lagrange Multiplier |
|---|---|---|---|
| 0.3858 | 0.4687 | 1 | 65.5261 |
| 0.4871 | 0.611 | −1 | 65.5261 |
| 0.9218 | 0.4103 | −1 | 0 |
| 0.7382 | 0.8936 | −1 | 0 |
| 0.1763 | 0.0579 | 1 | 0 |
| 0.4057 | 0.3529 | 1 | 0 |
| 0.9355 | 0.8132 | −1 | 0 |
| 0.2146 | 0.0099 | 1 | 0 |

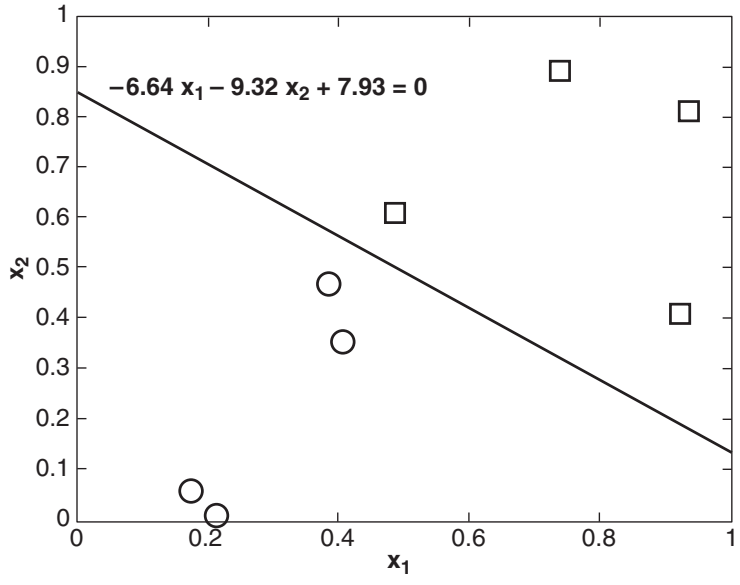



**Figure 6.34.** Example of a linearly separable data set.

The bias term $b$ can be computed using Equation 6.76 for each support vector:

$$b^{(1)} = 1 - \mathbf{w} \cdot \mathbf{x}_1 = 1 - (-6.64)(0.3858) - (-9.32)(0.4687) = 7.9300.$$
$$b^{(2)} = -1 - \mathbf{w} \cdot \mathbf{x}_2 = -1 - (-6.64)(0.4871) - (-9.32)(0.611) = 7.9289.$$

Averaging these values, we obtain $b = 7.93$. The decision boundary corresponding to these parameters is shown in Figure 6.34. ■

### 6.9.3 Soft-margin SVM

Figure 6.35 shows a data set that is similar to Figure 6.32, except it has two new examples, $P$ and $Q$. Although the decision boundary $B_1$ misclassifies the new examples, while $B_2$ classifies them correctly, this does not mean that $B_2$ is a better decision boundary than $B_1$ because the new examples may correspond to noise in the training data. $B_1$ should still be preferred over $B_2$ because it has a wider margin, and thus, is less susceptible to overfitting. However, the SVM formulation presented in the previous section only constructs decision boundaries that are mistake-free.

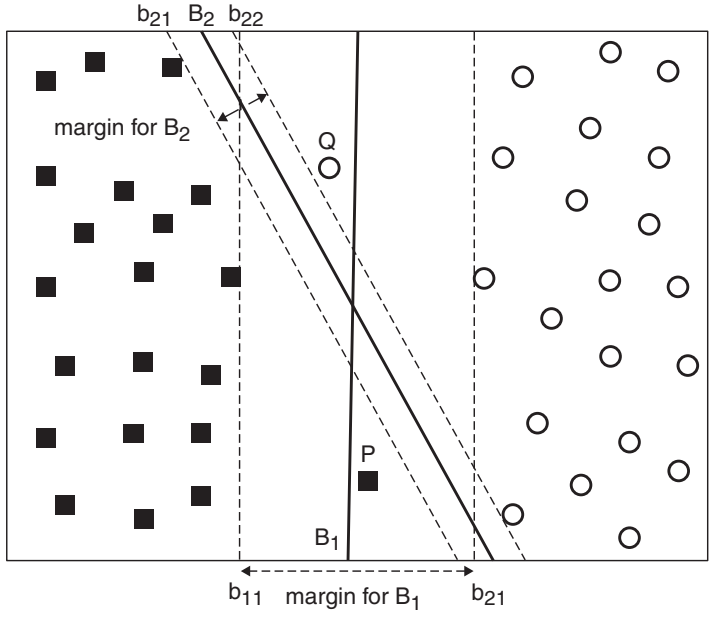



**Figure 6.35.** Decision boundary of SVM for the non-separable case.

This section examines how the formulation of SVM can be modified to learn a separating hyperplane that is tolerable to small number of training errors using a method known as the **soft-margin** approach. More importantly, the method presented in this section allows SVM to learn linear hyperplanes even in situations where the classes are not linearly separable. To do this, the learning algorithm in SVM must consider the trade-off between the width of the margin and the number of training errors committed by the linear hyperplane.

To introduce the concept of training errors in the SVM formulation, let us relax the inequality constraints to accommodate for some violations on a small number of training instances. This can be done by introducing a **slack variable** $\xi \geq 0$ for every training instance $\mathbf{x}_i$ as follows:

$$y_i(\mathbf{w}^T\mathbf{x}_i + b) \geq 1 - \xi_i \tag{6.80}$$

The variable $\xi_i$ allows for some slack in the inequalities of the SVM such that every instance $\mathbf{x}_i$ does not need to strictly satisfy $y_i(\mathbf{w}^T\mathbf{x}_i + b) \geq 1$. Further, $\xi_i$ is non-zero only if the margin hyperplanes are not able to place $\mathbf{x}_i$ on the same side as the rest of the instances belonging to $y_i$. To illustrate this, Figure 6.36 shows a circle $P$ that falls on the opposite side of the separating hyperplane as the rest of the circles, and thus satisfies $\mathbf{w}^T\mathbf{x} + b = -1 + \xi$. The distance between $P$ and the margin hyperplane $\mathbf{w}^T\mathbf{x} + b = -1$ is equal to $\xi/\|\mathbf{w}\|$. Hence, $\xi_i$ provides a measure of the error of SVM in representing $\mathbf{x}_i$ using *soft* inequality constraints.

In the presence of slack variables, it is important to learn a separating hyperplane that jointly maximizes the margin (ensuring good generalization performance) and minimizes the values of slack variables (ensuring low training error). This can be achieved by modifying the optimization problem of

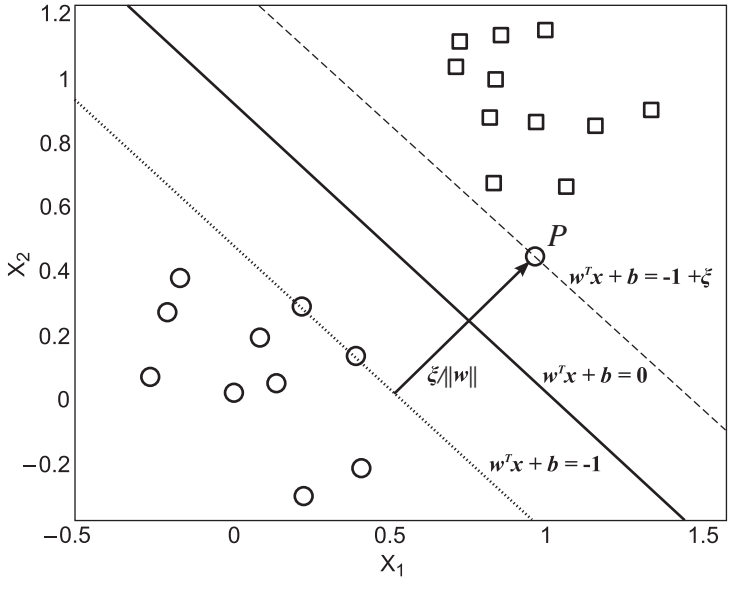



**Figure 6.36.** Slack variables used in soft-margin SVM.

SVM as follows:

$$\begin{aligned} & \min_{\mathbf{w},b,\xi_i} \frac{||\mathbf{w}||^2}{2} + C\sum_{i=1}^{n}\xi_i \\ \text{subject to} \quad & y_i(\mathbf{w}^T\mathbf{x}_i + b) \geq 1 - \xi_i, \\ & \xi_i \geq 0. \end{aligned} \tag{6.81}$$

where $C$ is a hyper-parameter that makes a trade-off between maximizing the margin and minimizing the training error. A large value of $C$ pays more emphasis on minimizing the training error than maximizing the margin. Notice the similarity of the previous equation with the generic formula of generalization error rate introduced in Section 3.4. Indeed, SVM provides a natural way to balance between model complexity and training error in order to maximize generalization performance.

To solve Equation 6.81 we apply the Lagrange multiplier method and convert the primal problem to its corresponding dual problem, similar to the approach described in the previous section. The Lagrangian primal problem of Equation 6.81 can be written as follows:

$$L_P = \frac{1}{2}\|\mathbf{w}\|^2 + C\sum_{i=1}^{n}\xi_i - \sum_{i=1}^{n}\lambda_i\Big(y_i(\mathbf{w}^T\mathbf{x}_i + b) - 1 + \xi_i\Big) - \sum_{i=1}^{n}\mu_i\Big(\xi_i\Big), \tag{6.82}$$

where $\lambda_i \geq 0$ and $\mu_i \geq 0$ are the Lagrange multipliers corresponding to the inequality constraints of Equation 6.81. Setting the derivative of $L_P$ with respect to $\mathbf{w}$, $b$, and $\xi_i$ equal to 0, we obtain the following equations:

$$\frac{\partial L_P}{\partial \mathbf{w}} = 0 \implies \mathbf{w} = \sum_{i=1}^{n}\lambda_i y_i \mathbf{x}_i. \tag{6.83}$$

$$\frac{\partial L}{\partial b} = 0 \implies \sum_{i=1}^{n}\lambda_i y_i = 0. \tag{6.84}$$

$$\frac{\partial L}{\partial \xi_i} = 0 \implies \lambda_i + \mu_i = C. \tag{6.85}$$

We can also obtain the complementary slackness conditions by using the following KKT conditions:

$$\lambda_i \left( y_i(\mathbf{w}^T \mathbf{x}_i + b) - 1 + \xi_i \right) = 0, \tag{6.86}$$

$$\mu_i \xi_i = 0. \tag{6.87}$$

Equation 6.86 suggests that $\lambda_i$ is zero for all training instances except those that reside on the margin hyperplanes $\mathbf{w}^T \mathbf{x}_i + b = \pm 1$, or have $\xi_i > 0$. These instances with $\lambda_i > 0$ are known as support vectors. On the other hand, $\mu_i$ given in Equation 6.87 is zero for any training instance that is misclassified, i.e., $\xi_i > 0$. Further, $\lambda_i$ and $\mu_i$ are related with each other by Equation 6.85. This results in the following three configurations of $(\lambda_i, \mu_i)$:

1. If $\lambda_i = 0$ and $\mu_i = C$, then $\mathbf{x}_i$ does not reside on the margin hyperplanes and is correctly classified on the same side as other instances belonging to $y_i$.

2. If $\lambda_i = C$ and $\mu_i = 0$, then $\mathbf{x}_i$ is misclassified and has a non-zero slack variable $\xi_i$.

3. If $0 < \lambda_i < C$ and $0 < \mu_i < C$, then $\mathbf{x}_i$ resides on one of the margin hyperplanes.

Substituting Equations 6.83 to 6.87 into Equation 6.82, we obtain the following dual optimization problem:

$$\begin{aligned} \max_{\lambda_i} \quad & \sum_{i=1}^{n} \lambda_i - \frac{1}{2} \sum_{i=1}^{n} \sum_{j=1}^{n} \lambda_i \lambda_j y_i y_j \mathbf{x}_i^T \mathbf{x}_j \\ \text{subject to} \quad & \sum_{i=1}^{n} \lambda_i y_i = 0, \\ & 0 \leq \lambda_i \leq C. \end{aligned} \tag{6.88}$$

Notice that the previous problem looks almost identical to the dual problem of SVM for the linearly separable case (Equation 6.77), except that $\lambda_i$ is required to not only be greater than 0 but also smaller than a constant value $C$. Clearly, when $C$ reaches infinity, the previous optimization problem becomes equivalent to Equation 6.77, where the learned hyperplane perfectly separates the classes (with no training errors). However, by capping the values of $\lambda_i$ to $C$, the learned hyperplane is able to tolerate a few training errors that have $\xi_i > 0$.

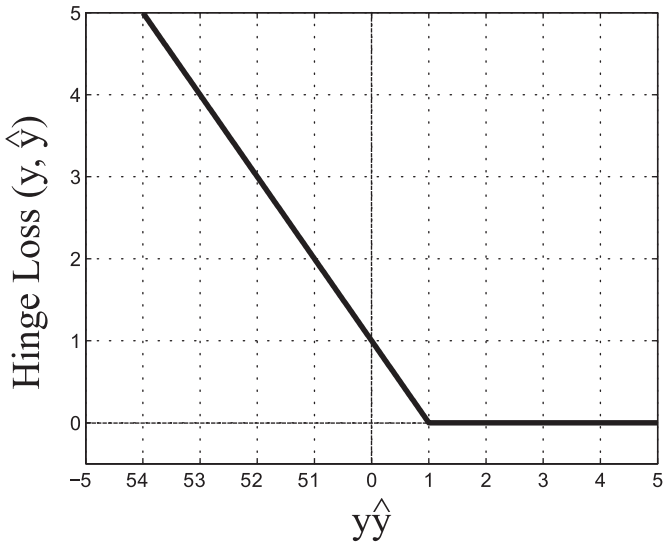



**Figure 6.37.** Hinge loss as a function of $y\hat{y}$.

As before, Equation 6.88 can be solved by using any of the standard solvers for QPP, and the optimal value of $\mathbf{w}$ can be obtained by using Equation 6.83. To solve for $b$, we can use Equation 6.86 on the support vectors that reside on the margin hyperplanes as follows:

$$b = \frac{1}{n_S} \sum_{i \in S} \frac{1 - y_i \mathbf{w}^T \mathbf{x}_i}{y_i} \tag{6.89}$$

where $S$ represents the set of support vectors residing on the margin hyperplanes ($S = \{i | 0 < \lambda_i < C\}$) and $n_S$ is the number of elements in $S$.

### SVM as a Regularizer of Hinge Loss

SVM belongs to a broad class of regularization techniques that use a loss function to represent the training errors and a norm of the model parameters to represent the model complexity. To realize this, notice that the slack variable $\xi$, used for measuring the training errors in SVM, is equivalent to the hinge loss function, which can be defined as follows:

$$\text{Loss } (y, \hat{y}) = \max \ (0, 1 - y\hat{y}),$$

where $y \in \{+1, -1\}$. In the case of SVM, $\hat{y}$ corresponds to $\mathbf{w}^T\mathbf{x} + b$. Figure 6.37 shows a plot of the hinge loss function as we vary $y\hat{y}$. We can see that

the hinge loss is equal to 0 as long as $y$ and $\hat{y}$ have the same sign and $|\hat{y}| \geq 1$. However, the hinge loss grows linearly with $|\hat{y}|$ whenever $y$ and $\hat{y}$ are of the opposite sign or $|\hat{y}| < 1$. This is similar to the notion of the slack variable $\xi$, which is used to measure the distance of a point from its margin hyperplane. Hence, the optimization problem of SVM can be represented in the following equivalent form:

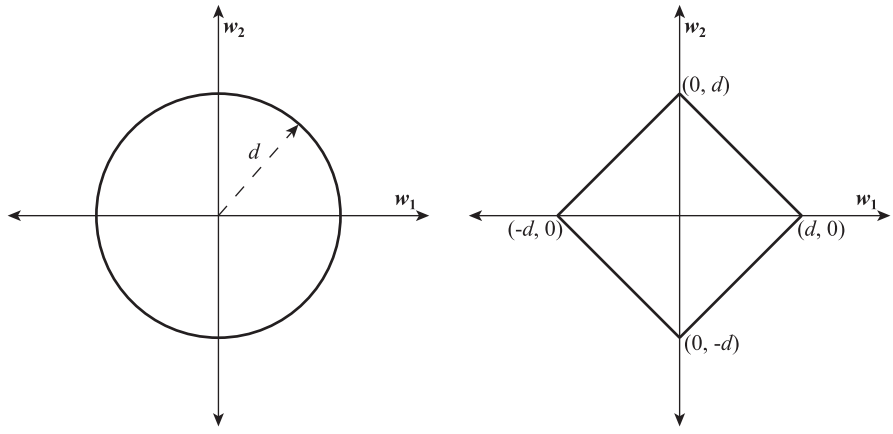
$$\min_{\mathbf{w},b} \frac{||\mathbf{w}||^2}{2} + C \sum_{i=1}^{n} \text{Loss } (y_i, \mathbf{w}^T \mathbf{x}_i + b) \tag{6.90}$$

Note that using the hinge loss ensures that the optimization problem is convex and can be solved using standard optimization techniques. However, if we use a different loss function, such as the squared loss function that was introduced in Section 6.7 on ANN, it will result in a different optimization problem that may or may not remain convex. Nevertheless, different loss functions can be explored to capture varying notions of training error, depending on the characteristics of the problem.

Another interesting property of SVM that relates it to a broader class of regularization techniques is the concept of a margin. Although minimizing $||\mathbf{w}||^2$ has the geometric interpretation of maximizing the margin of a separating hyperplane, it is essentially the squared $L_2$ norm of the model parameters, $||\mathbf{w}||_2^2$. In general, the $L_q$ norm of $\mathbf{w}$, $||\mathbf{w}||_q$, is equal to the Minkowski distance of order $q$ from $\mathbf{w}$ to the origin, i.e.,

$$||\mathbf{w}||_q = \left( \sum_{i}^{p} w_i^q \right)^{1/q}$$

Minimizing the $L_q$ norm of $\mathbf{w}$ to achieve lower model complexity is a generic regularization concept that has several interpretations. For example, minimizing the $L_2$ norm amounts to finding a solution on a hypersphere of smallest radius that shows suitable training performance. To visualize this in two-dimensions, Figure 6.38(a) shows the plot of a circle with constant radius $r$, where every point has the same $L_2$ norm. On the other hand, using the $L_1$ norm ensures that the solution lies on the surface of a hypercube with smallest size, with vertices along the axes. This is illustrated in Figure 6.38(b) as a square with vertices on the axes at a distance of $r$ from the origin. The $L_1$ norm is commonly used as a regularizer to obtain *sparse* model parameters with only a small number of non-zero parameter values, such as the use of Lasso in regression problems (see Bibliographic Notes).



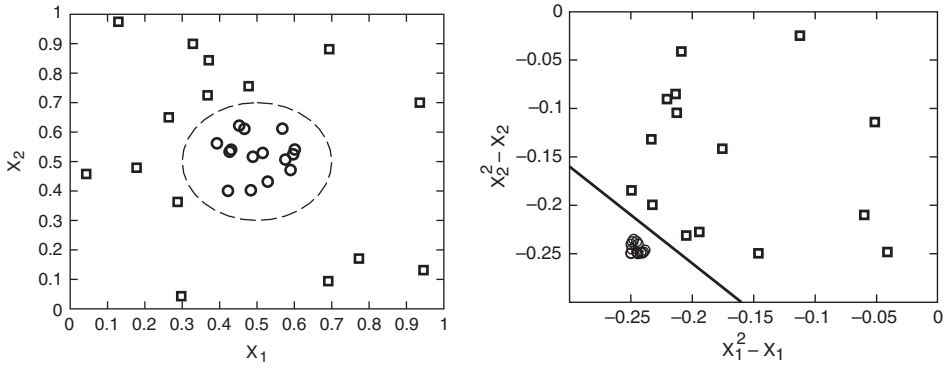

(a) $L_2$ norm. (b) $L_1$ norm.

**Figure 6.38.** Plots showing the behavior of two-dimensional solutions with constant $L_2$ and $L_1$ norms.

In general, depending on the characteristics of the problem, different combinations of $L_q$ norms and training loss functions can be used for learning the model parameters, each requiring a different optimization solver. This forms the backbone of a wide range of modeling techniques that attempt to improve the generalization performance by jointly minimizing training error and model complexity. However, in this section, we focus only on the squared $L_2$ norm and the hinge loss function, resulting in the classical formulation of SVM.

### 6.9.4 Nonlinear SVM

The SVM formulations described in the previous sections construct a linear decision boundary to separate the training examples into their respective classes. This section presents a methodology for applying SVM to data sets that have nonlinear decision boundaries. The basic idea is to transform the data from its original attribute space in $\mathbf{x}$ into a new space $\varphi(\mathbf{x})$ so that a linear hyperplane can be used to separate the instances in the transformed space, using the SVM approach. The learned hyperplane can then be projected back to the original attribute space, resulting in a nonlinear decision boundary.




(a) Decision boundary in the original two-dimensional space.

(b) Decision boundary in the transformed space.

**Figure 6.39.** Classifying data with a nonlinear decision boundary.

### Attribute Transformation

To illustrate how attribute transformation can lead to a linear decision boundary, Figure 6.39(a) shows an example of a two-dimensional data set consisting of squares (classified as $y = 1$) and circles (classified as $y = -1$). The data set is generated in such a way that all the circles are clustered near the center of the diagram and all the squares are distributed farther away from the center. Instances of the data set can be classified using the following equation:

$$y = \begin{cases} 1 & \text{if } \sqrt{(x_1 - 0.5)^2 + (x_2 - 0.5)^2} > 0.2, \\ -1 & \text{otherwise.} \end{cases} \tag{6.91}$$

The decision boundary for the data can therefore be written as follows:

$$\sqrt{(x_1 - 0.5)^2 + (x_2 - 0.5)^2} = 0.2,$$

which can be further simplified into the following quadratic equation:

$$x_1^2 - x_1 + x_2^2 - x_2 = -0.46.$$

A nonlinear transformation $\varphi$ is needed to map the data from its original attribute space into a new space such that a linear hyperplane can separate

the classes. This can be achieved by using the following simple transformation:

$$\varphi : (x_1, x_2) \longrightarrow (x_1^2 - x_1, x_2^2 - x_2). \tag{6.92}$$

Figure 6.39(b) shows the points in the transformed space, where we can see that all the circles are located in the lower left-hand side of the diagram. A linear hyperplane with parameters $\mathbf{w}$ and $b$ can therefore be constructed in the transformed space, to separate the instances into their respective classes.

One may think that because the nonlinear transformation possibly increases the dimensionality of the input space, this approach can suffer from the curse of dimensionality that is often associated with high-dimensional data. However, as we will see in the following section, nonlinear SVM is able to avoid this problem by using kernel functions.

### Learning a Nonlinear SVM Model

Using a suitable function, $\varphi(.)$, we can transform any data instance $\mathbf{x}$ to $\varphi(\mathbf{x})$. (The details on how to choose $\varphi(.)$ will become clear later.) The linear hyperplane in the transformed space can be expressed as $\mathbf{w}^T\varphi(\mathbf{x}) + b = 0$. To learn the optimal separating hyperplane, we can substitute $\varphi(\mathbf{x})$ for $\mathbf{x}$ in the formulation of SVM to obtain the following optimization problem:

$$\begin{aligned} & \min_{\mathbf{w},b,\xi_i} \frac{||\mathbf{w}||^2}{2} + C\sum_{i=1}^{n}\xi_i \\ \text{subject to} \quad & y_i(\mathbf{w}^T\varphi(\mathbf{x}_i) + b) \geq 1 - \xi_i, \\ & \xi_i \geq 0. \end{aligned} \tag{6.93}$$

Using Lagrange multipliers $\lambda_i$, this can be converted into a dual optimization problem:

$$\begin{aligned} \max_{\lambda_i} \quad & \sum_{i=1}^{n}\lambda_i - \frac{1}{2}\sum_{i=1}^{n}\sum_{j=1}^{n}\lambda_i\lambda_j y_i y_j \langle\varphi(\mathbf{x}_i), \varphi(\mathbf{x}_j)\rangle \\ \text{subject to} \quad & \sum_{i=1}^{n}\lambda_i y_i = 0, \\ & 0 \leq \lambda_i \leq C, \end{aligned} \tag{6.94}$$

where $\langle \mathbf{a}, \mathbf{b}\rangle$ denotes the inner product between vectors $\mathbf{a}$ and $\mathbf{b}$. Also, the equation of the hyperplane in the transformed space can be represented using

$\lambda_i$ as follows:

$$\sum_{i=1}^{n} \lambda_i y_i \langle \varphi(\mathbf{x}_i), \varphi(\mathbf{x}) \rangle + b = 0. \tag{6.95}$$

Further, $b$ is given by

$$b = \frac{1}{n_S} \left( \sum_{i \in S} \frac{1}{y_i} - \sum_{i \in S} \sum_{j=1}^{n} \frac{\lambda_j y_i y_j \langle \varphi(\mathbf{x}_i), \varphi(\mathbf{x}_j) \rangle}{y_i} \right), \tag{6.96}$$

where $S = \{i | 0 < \lambda_i < C\}$ is the set of support vectors residing on the margin hyperplanes and $n_S$ is the number of elements in $S$.

Note that in order to solve the dual optimization problem in Equation 6.94, or to use the learned model parameters to make predictions using Equations 6.95 and 6.96, we need only inner products of $\varphi(\mathbf{x})$. Hence, even though $\varphi(\mathbf{x})$ may be nonlinear and high-dimensional, it suffices to use a function of the inner products of $\varphi(\mathbf{x})$ in the transformed space. This can be achieved by using a kernel trick, which can be described as follows.

The inner product between two vectors is often regarded as a measure of similarity between the vectors. For example, the cosine similarity described in Section 2.4.5 on page 99 can be defined as the dot product between two vectors that are normalized to unit length. Analogously, the inner product $\langle \varphi(\mathbf{x}_i), \varphi(\mathbf{x}_j) \rangle$ can also be regarded as a measure of similarity between two instances, $\mathbf{x}_i$ and $\mathbf{x}_j$, in the transformed space. The **kernel trick** is a method for computing this similarity as a function of the original attributes. Specifically, the kernel function $K(\mathbf{u}, \mathbf{v})$ between two instances $u$ and $v$ can be defined as follows:

$$K(\mathbf{u}, \mathbf{v}) = \langle \varphi(\mathbf{u}), \varphi(\mathbf{v}) \rangle = f(\mathbf{u}, \mathbf{v}). \tag{6.97}$$

where $f(.)$ is a function that follows certain conditions as stated by the Mercer's Theorem. Although the details of this theorem are outside the scope of the book, we provide a list of some of the commonly used kernel functions:

$$\text{Polynomial kernel} \qquad K(\mathbf{u}, \mathbf{v}) = (\mathbf{u}^T \mathbf{v} + 1)^p \tag{6.98}$$

$$\text{Radial Basis Function kernel} \qquad K(\mathbf{u}, \mathbf{v}) = e^{-\|\mathbf{u}-\mathbf{v}\|^2/(2\sigma^2)} \tag{6.99}$$

$$\text{Sigmoid kernel} \qquad K(\mathbf{u}, \mathbf{v}) = \tanh(k\mathbf{u}^T \mathbf{v} - \delta) \tag{6.100}$$

By using a kernel function, we can directly work with inner products in the transformed space without dealing with the exact forms of the nonlinear

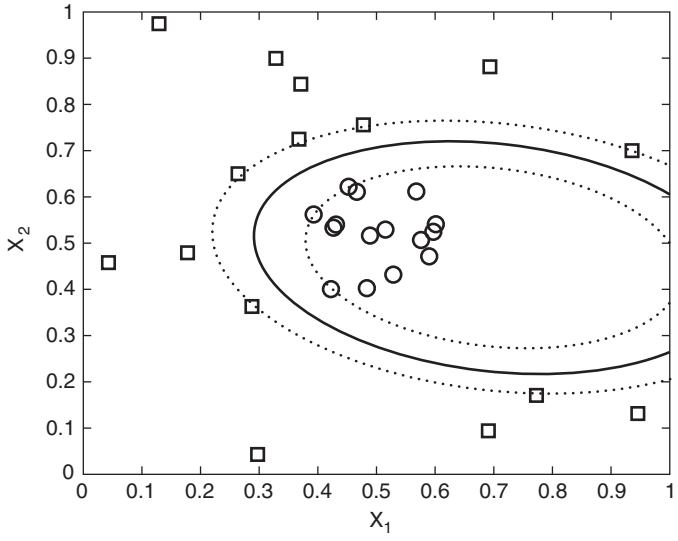



**Figure 6.40.** Decision boundary produced by a nonlinear SVM with polynomial kernel.

transformation function $\varphi$. Specifically, this allows us to use high-dimensional transformations (sometimes even involving infinitely many dimensions), while performing calculations only in the original attribute space. Computing the inner products using kernel functions is also considerably cheaper than using the transformed attribute set $\varphi(\mathbf{x})$. Hence, the use of kernel functions provides a significant advantage in representing nonlinear decision boundaries, without suffering from the curse of dimensionality. This has been one of the major reasons behind the widespread usage of SVM in highly complex and nonlinear problems.

Figure 6.40 shows the nonlinear decision boundary obtained by SVM using the polynomial kernel function given in Equation 6.98. We can see that the learned decision boundary is quite close to the true decision boundary shown in Figure 6.39(a). Although the choice of kernel function depends on the characteristics of the input data, a commonly used kernel function is the radial basis function (RBF) kernel, which involves a single hyper-parameter $\sigma$, known as the standard deviation of the RBF kernel.

### 6.9.5 Characteristics of SVM

1. The SVM learning problem can be formulated as a convex optimization problem, in which efficient algorithms are available to find the global minimum of the objective function. Other classification methods, such

as rule-based classifiers and artificial neural networks, employ a greedy strategy to search the hypothesis space. Such methods tend to find only locally optimum solutions.

2. SVM provides an effective way of regularizing the model parameters by maximizing the margin of the decision boundary. Furthermore, it is able to create a balance between model complexity and training errors by using a hyper-parameter $C$. This trade-off is generic to a broader class of model learning techniques that capture the model complexity and the training loss using different formulations.

3. Linear SVM can handle irrelevant attributes by learning zero weights corresponding to such attributes. It can also handle redundant attributes by learning similar weights for the duplicate attributes. Furthermore, the ability of SVM to regularize its learning makes it more robust to the presence of a large number of irrelevant and redundant attributes than other classifiers, even in high-dimensional settings. For this reason, nonlinear SVMs are less impacted by irrelevant and redundant attributes than other highly expressive classifiers that can learn nonlinear decision boundaries such as decision trees.

   To compare the effect of irrelevant attributes on the performance of nonlinear SVMs and decision trees, consider the two-dimensional data set shown in Figure 6.41(a) containing 500 + and 500 o instances, where the two classes can be easily separated using a nonlinear decision boundary. We incrementally add irrelevant attributes to this data set and compare the performance of two classifiers: decision tree and nonlinear SVM (using radial basis function kernel), using 70% of the data for training and the rest for testing. Figure 6.41(b) shows the test error rates of the two classifiers as we increase the number of irrelevant attributes. We can see that the test error rate of decision trees swiftly reaches 0.5 (same as random guessing) in the presence of even a small number of irrelevant attributes. This can be attributed to the problem of multiple comparisons while choosing splitting attributes at internal nodes as discussed in Example 3.7. On the other hand, nonlinear SVM shows a more robust and steady performance even after adding a moderately large number of irrelevant attributes. Its test error rate gradually declines and eventually reaches close to 0.5 after adding 125 irrelevant attributes, at which point it becomes difficult to discern the discriminative information in the original two attributes from the noise in the remaining attributes for learning nonlinear decision boundaries.

test error rate remains close to 0.5 even after using 50 leaf nodes, while the training error rate keeps on declining and eventually becomes 0.

■

## 3.5 Model Selection

There are many possible classification models with varying levels of model complexity that can be used to capture patterns in the training data. Among these possibilities, we want to select the model that shows lowest generalization error rate. The process of selecting a model with the right level of complexity, which is expected to generalize well over unseen test instances, is known as **model selection**. As described in the previous section, the training error rate cannot be reliably used as the sole criterion for model selection. In the following, we present three generic approaches to estimate the generalization performance of a model that can be used for model selection. We conclude this section by presenting specific strategies for using these approaches in the context of decision tree induction.

### 3.5.1 Using a Validation Set

Note that we can always estimate the generalization error rate of a model by using "out-of-sample" estimates, i.e. by evaluating the model on a separate **validation set** that is not used for training the model. The error rate on the validation set, termed as the validation error rate, is a better indicator of generalization performance than the training error rate, since the validation set has not been used for training the model. The validation error rate can be used for model selection as follows.

Given a training set $D.train$, we can partition $D.train$ into two smaller subsets, $D.tr$ and $D.val$, such that $D.tr$ is used for training while $D.val$ is used as the validation set. For example, two-thirds of $D.train$ can be reserved as $D.tr$ for training, while the remaining one-third is used as $D.val$ for computing validation error rate. For any choice of classification model $m$ that is trained on $D.tr$, we can estimate its validation error rate on $D.val$, $err_{val}(m)$. The model that shows the lowest value of $err_{val}(m)$ can then be selected as the preferred choice of model.

The use of validation set provides a generic approach for model selection. However, one limitation of this approach is that it is sensitive to the sizes of $D.tr$ and $D.val$, obtained by partitioning $D.train$. If the size of $D.tr$ is too small, it may result in the learning of a poor classification model with sub-standard performance, since a smaller training set will be less representative

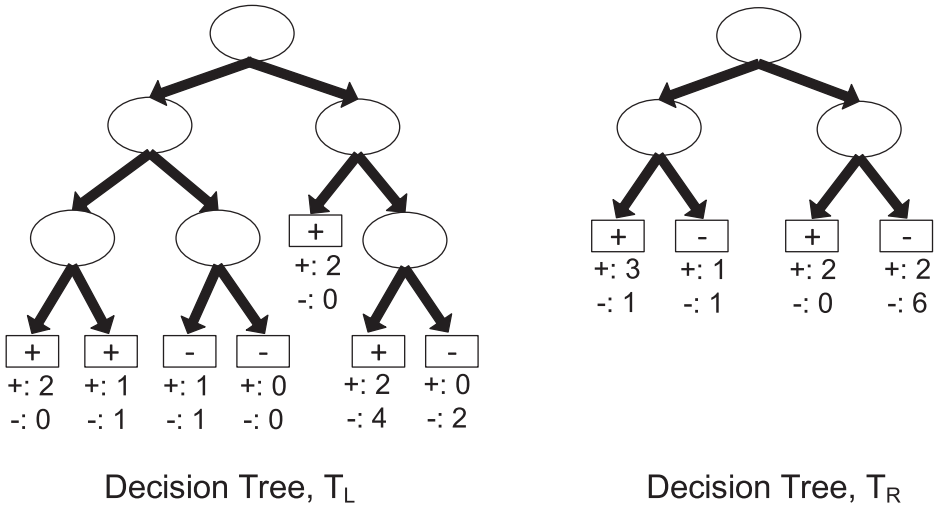



**Figure 3.29.** Class distribution of validation data for the two decision trees shown in Figure 3.30.

of the overall data. On the other hand, if the size of $D.val$ is too small, the validation error rate might not be reliable for selecting models, as it would be computed over a small number of instances.

**Example 3.8. [Validation Error]** In the following example, we illustrate one possible approach for using a validation set in decision tree induction. Figure 3.29 shows the predicted labels at the leaf nodes of the decision trees generated in Figure 3.30. The counts given beneath the leaf nodes represent the proportion of data instances in the validation set that reach each of the nodes. Based on the predicted labels of the nodes, the validation error rate for the left tree is $err_{val}(T_L) = 6/16 = 0.375$, while the validation error rate for the right tree is $err_{val}(T_R) = 4/16 = 0.25$. Based on their validation error rates, the right tree is preferred over the left one. ■

### 3.5.2 Incorporating Model Complexity

Since the chance for model overfitting increases as the model becomes more complex, a model selection approach should not only consider the training error rate but also the model complexity. This strategy is inspired by a well-known principle known as **Occam's razor** or the **principle of parsimony**, which suggests that given two models with the same errors, the simpler model is preferred over the more complex model. A generic approach to account

for model complexity while estimating generalization performance is formally described as follows.

Given a training set $D.train$, let us consider learning a classification model $m$ that belongs to a certain class of models, $\mathcal{M}$. For example, if $\mathcal{M}$ represents the set of all possible decision trees, then $m$ can correspond to a specific decision tree learned from the training set. We are interested in estimating the generalization error rate of $m$, $gen.error(m)$. As discussed previously, the training error rate of $m$, $train.error(m, D.train)$, can under-estimate $gen.error(m)$ when the model complexity is high. Hence, we represent $gen.error(m)$ as a function of not just the training error rate but also the model complexity of $\mathcal{M}$, $complexity(\mathcal{M})$, as follows:

$$gen.error(m) = train.error(m, D.train) + \alpha \times complexity(\mathcal{M}), \qquad (3.11)$$

where $\alpha$ is a hyper-parameter that strikes a balance between minimizing training error and reducing model complexity. A higher value of $\alpha$ gives more emphasis to the model complexity in the estimation of generalization performance. To choose the right value of $\alpha$, we can make use of the validation set in a similar way as described in 3.5.1. For example, we can iterate through a range of values of $\alpha$ and for every possible value, we can learn a model on a subset of the training set, $D.tr$, and compute its validation error rate on a separate subset, $D.val$. We can then select the value of $\alpha$ that provides the lowest validation error rate.

Equation 3.11 provides one possible approach for incorporating model complexity into the estimate of generalization performance. This approach is at the heart of a number of techniques for estimating generalization performance, such as the structural risk minimization principle, the Akaike's Information Criterion (AIC), and the Bayesian Information Criterion (BIC). The structural risk minimization principle serves as the building block for learning support vector machines, which will be discussed later in Chapter 6. For more details on AIC and BIC, see the Bibliographic Notes.

In the following, we present two different approaches for estimating the complexity of a model, $complexity(\mathcal{M})$. While the former is specific to decision trees, the latter is more generic and can be used with any class of models.

### Estimating the Complexity of Decision Trees

In the context of decision trees, the complexity of a decision tree can be estimated as the ratio of the number of leaf nodes to the number of training instances. Let $k$ be the number of leaf nodes and $N_{train}$ be the number of

training instances. The complexity of a decision tree can then be described as $k/N_{train}$. This reflects the intuition that for a larger training size, we can learn a decision tree with larger number of leaf nodes without it becoming overly complex. The generalization error rate of a decision tree $T$ can then be computed using Equation 3.11 as follows:

$$err_{gen}(T) = err(T) + \Omega \times \frac{k}{N_{train}},$$

where $err(T)$ is the training error of the decision tree and $\Omega$ is a hyper-parameter that makes a trade-off between reducing training error and minimizing model complexity, similar to the use of $\alpha$ in Equation 3.11. $\Omega$ can be viewed as the relative cost of adding a leaf node relative to incurring a training error. In the literature on decision tree induction, the above approach for estimating generalization error rate is also termed as the **pessimistic error estimate**. It is called pessimistic as it assumes the generalization error rate to be worse than the training error rate (by adding a penalty term for model complexity). On the other hand, simply using the training error rate as an estimate of the generalization error rate is called the **optimistic error estimate** or the **resubstitution estimate**.

**Example 3.9. [Generalization Error Estimates]** Consider the two binary decision trees, $T_L$ and $T_R$, shown in Figure 3.30. Both trees are generated from the same training data and $T_L$ is generated by expanding three leaf nodes of $T_R$. The counts shown in the leaf nodes of the trees represent the class

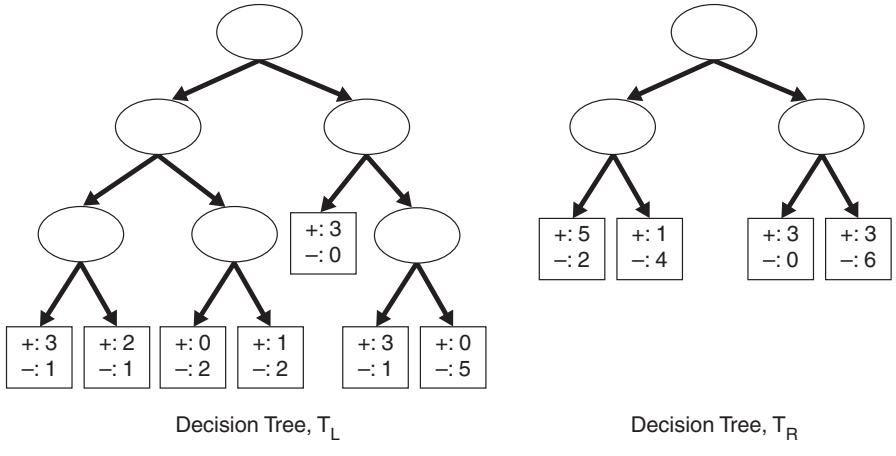



**Figure 3.30.** Example of two decision trees generated from the same training data.

distribution of the training instances. If each leaf node is labeled according to the majority class of training instances that reach the node, the training error rate for the left tree will be $err(T_L) = 4/24 = 0.167$, while the training error rate for the right tree will be $err(T_R) = 6/24 = 0.25$. Based on their training error rates alone, $T_L$ would preferred over $T_R$, even though $T_L$ is more complex (contains larger number of leaf nodes) than $T_R$.

Now, assume that the cost associated with each leaf node is $\Omega = 0.5$. Then, the generalization error estimate for $T_L$ will be

$$err_{gen}(T_L) = \frac{4}{24} + 0.5 \times \frac{7}{24} = \frac{7.5}{24} = 0.3125$$

and the generalization error estimate for $T_R$ will be

$$err_{gen}(T_R) = \frac{6}{24} + 0.5 \times \frac{4}{24} = \frac{8}{24} = 0.3333.$$

Since $T_L$ has a lower generalization error rate, it will still be preferred over $T_R$. Note that $\Omega = 0.5$ implies that a node should always be expanded into its two child nodes if it improves the prediction of at least one training instance, since expanding a node is less costly than misclassifying a training instance. On the other hand, if $\Omega = 1$, then the generalization error rate for $T_L$ is $err_{gen}(T_L) = 11/24 = 0.458$ and for $T_R$ is $err_{gen}(T_R) = 10/24 = 0.417$. In this case, $T_R$ will be preferred over $T_L$ because it has a lower generalization error rate. This example illustrates that different choices of $\Omega$ can change our preference of decision trees based on their generalization error estimates. However, for a given choice of $\Omega$, the pessimistic error estimate provides an approach for modeling the generalization performance on unseen test instances. The value of $\Omega$ can be selected with the help of a validation set. ■

### Minimum Description Length Principle

Another way to incorporate model complexity is based on an information-theoretic approach known as the minimum description length or MDL principle. To illustrate this approach, consider the example shown in Figure 3.31. In this example, both person `A` and person `B` are given a set of instances with known attribute values $\mathbf{x}$. Assume person `A` knows the class label $y$ for every instance, while person `B` has no such information. `A` would like to share the class information with `B` by sending a message containing the labels. The message would contain $\Theta(N)$ bits of information, where $N$ is the number of instances.

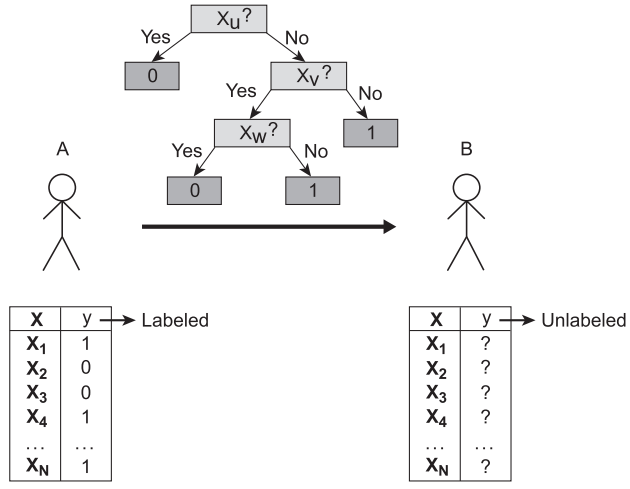



**Figure 3.31.** An illustration of the minimum description length principle.

Alternatively, instead of sending the class labels explicitly, `A` can build a classification model from the instances and transmit it to `B`. `B` can then apply the model to determine the class labels of the instances. If the model is 100% accurate, then the cost for transmission is equal to the number of bits required to encode the model. Otherwise, `A` must also transmit information about which instances are misclassified by the model so that `B` can reproduce the same class labels. Thus, the overall transmission cost, which is equal to the total description length of the message, is

$$Cost(model, data) = Cost(data|model) + \alpha \times Cost(model), \tag{3.12}$$

where the first term on the right-hand side is the number of bits needed to encode the misclassified instances, while the second term is the number of bits required to encode the model. There is also a hyper-parameter $\alpha$ that trades-off the relative costs of the misclassified instances and the model. Notice the familiarity between this equation and the generic equation for generalization error rate presented in Equation 3.11. A good model must have a total description length less than the number of bits required to encode the entire sequence of class labels. Furthermore, given two competing models, the model with lower total description length is preferred. An example showing how to compute the total description length of a decision tree is given in Exercise 11 on page 209.

### 3.5.3 Estimating Statistical Bounds

Instead of using Equation 3.11 to estimate the generalization error rate of a model, an alternative way is to apply a statistical correction to the training error rate of the model that is indicative of its model complexity. This can be done if the probability distribution of training error is available or can be assumed. For example, the number of errors committed by a leaf node in a decision tree can be assumed to follow a binomial distribution. We can thus compute an upper bound limit to the observed training error rate that can be used for model selection, as illustrated in the following example.

**Example 3.10. [Statistical Bounds on Training Error]** Consider the left-most branch of the binary decision trees shown in Figure 3.30. Observe that the left-most leaf node of $T_R$ has been expanded into two child nodes in $T_L$. Before splitting, the training error rate of the node is $2/7 = 0.286$. By approximating a binomial distribution with a normal distribution, the following upper bound of the training error rate $e$ can be derived:

$$e_{upper}(N, e, \alpha) = \frac{e + \frac{z^2_{\alpha/2}}{2N} + z_{\alpha/2}\sqrt{\frac{e(1-e)}{N} + \frac{z^2_{\alpha/2}}{4N^2}}}{1 + \frac{z^2_{\alpha/2}}{N}}, \tag{3.13}$$

where $\alpha$ is the confidence level, $z_{\alpha/2}$ is the standardized value from a standard normal distribution, and $N$ is the total number of training instances used to compute $e$. By replacing $\alpha = 25\%$, $N = 7$, and $e = 2/7$, the upper bound for the error rate is $e_{upper}(7, 2/7, 0.25) = 0.503$, which corresponds to $7 \times 0.503 = 3.521$ errors. If we expand the node into its child nodes as shown in $T_L$, the training error rates for the child nodes are $1/4 = 0.250$ and $1/3 = 0.333$, respectively. Using Equation (3.13), the upper bounds of these error rates are $e_{upper}(4, 1/4, 0.25) = 0.537$ and $e_{upper}(3, 1/3, 0.25) = 0.650$, respectively. The overall training error of the child nodes is $4 \times 0.537 + 3 \times 0.650 = 4.098$, which is larger than the estimated error for the corresponding node in $T_R$, suggesting that it should not be split. ■

### 3.5.4 Model Selection for Decision Trees

Building on the generic approaches presented above, we present two commonly used model selection strategies for decision tree induction.

**Prepruning (Early Stopping Rule)** In this approach, the tree-growing algorithm is halted before generating a fully grown tree that perfectly fits

the entire training data. To do this, a more restrictive stopping condition must be used; e.g., stop expanding a leaf node when the observed gain in the generalization error estimate falls below a certain threshold. This estimate of the generalization error rate can be computed using any of the approaches presented in the preceding three subsections, e.g., by using pessimistic error estimates, by using validation error estimates, or by using statistical bounds. The advantage of prepruning is that it avoids the computations associated with generating overly complex subtrees that overfit the training data. However, one major drawback of this method is that, even if no significant gain is obtained using one of the existing splitting criterion, subsequent splitting may result in better subtrees. Such subtrees would not be reached if prepruning is used because of the greedy nature of decision tree induction.

**Post-pruning** In this approach, the decision tree is initially grown to its maximum size. This is followed by a tree-pruning step, which proceeds to trim the fully grown tree in a bottom-up fashion. Trimming can be done by replacing a subtree with (1) a new leaf node whose class label is determined from the majority class of instances affiliated with the subtree (approach known as **subtree replacement**), or (2) the most frequently used branch of the subtree (approach known as **subtree raising**). The tree-pruning step terminates when no further improvement in the generalization error estimate is observed beyond a certain threshold. Again, the estimates of generalization error rate can be computed using any of the approaches presented in the previous three subsections. Post-pruning tends to give better results than prepruning because it makes pruning decisions based on a fully grown tree, unlike prepruning, which can suffer from premature termination of the tree-growing process. However, for post-pruning, the additional computations needed to grow the full tree may be wasted when the subtree is pruned.

Figure 3.32 illustrates the simplified decision tree model for the web robot detection example given in Section 3.3.5. Notice that the subtree rooted at `depth` $= 1$ has been replaced by one of its branches corresponding to `breadth` $<= 7$, `width` $> 3$, and `MultiP` $= 1$, using subtree raising. On the other hand, the subtree corresponding to `depth` $> 1$ and `MultiAgent` $= 0$ has been replaced by a leaf node assigned to class 0, using subtree replacement. The subtree for `depth` $> 1$ and `MultiAgent` $= 1$ remains intact.

folds ($N(k-1)/k$). This is different than the $err_{test}$ computed in the holdout method, which exactly corresponds to the specific model learned over $D.train$. Hence, although effectively utilizing every data instance in $D$ for training and testing, the $err_{test}$ computed in the cross-validation method does not represent the performance of a single model learned over a specific $D.train$.

Nonetheless, in practice, $err_{test}$ is typically used as an estimate of the generalization error of a model built on $D$. One motivation for this is that when the size of the training folds is closer to the size of the overall data (when $k$ is large), then $err_{test}$ resembles the expected performance of a model learned over a data set of the same size as $D$. For example, when $k$ is 10, every training fold is 90% of the overall data. The $err_{test}$ then should approach the expected performance of a model learned over 90% of the overall data, which will be close to the expected performance of a model learned over $D$.

## 3.7 Presence of Hyper-parameters

Hyper-parameters are parameters of learning algorithms that need to be determined before learning the classification model. For instance, consider the hyper-parameter $\alpha$ that appeared in Equation 3.11, which is repeated here for convenience. This equation was used for estimating the generalization error for a model selection approach that used an explicit representations of model complexity. (See Section 3.5.2.)

$$gen.error(m) = train.error(m, D.train) + \alpha \times complexity(\mathcal{M})$$

For other examples of hyper-parameters, see Chapter 6.

Unlike regular model parameters, such as the test conditions in the internal nodes of a decision tree, hyper-parameters such as $\alpha$ do not appear in the final classification model that is used to classify unlabeled instances. However, the values of hyper-parameters need to be determined during model selection—a process known as **hyper-parameter selection**—and must be taken into account during model evaluation. Fortunately, both tasks can be effectively accomplished via slight modifications of the cross-validation approach described in the previous section.

### 3.7.1 Hyper-parameter Selection

In Section 3.5.2, a validation set was used to select $\alpha$ and this approach is generally applicable for hyper-parameter section. Let $p$ be the hyper-parameter that needs to be selected from a finite range of values, $\mathcal{P} = \{p_1, p_2, \ldots p_n\}$.

Partition $D.train$ into $D.tr$ and $D.val$. For every choice of hyper-parameter value $p_i$, we can learn a model $m_i$ on $D.tr$, and apply this model on $D.val$ to obtain the validation error rate $err_{val}(p_i)$. Let $p^*$ be the hyper-parameter value that provides the lowest validation error rate. We can then use the model $m^*$ corresponding to $p^*$ as the final choice of classification model.

The above approach, although useful, uses only a subset of the data, $D.train$, for training and a subset, $D.val$, for validation. The framework of cross-validation, presented in Section 3.6.2, addresses both of those issues, albeit in the context of model evaluation. Here we indicate how to use a cross-validation approach for hyper-parameter selection. To illustrate this approach, let us partition $D.train$ into three folds as shown in Figure 3.34. At every run, one of the folds is used as $D.val$ for validation, and the remaining two folds are used as $D.tr$ for learning a model, for every choice of hyper-parameter value $p_i$. The overall validation error rate corresponding to each $p_i$ is computed by summing the errors across all the three folds. We then select the hyper-parameter value $p^*$ that provides the lowest validation error rate, and use it to learn a model $m^*$ on the entire training set $D.train$.

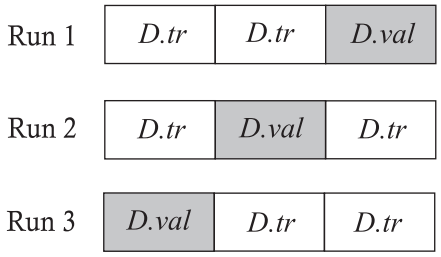



**Figure 3.34.** Example demonstrating the 3-fold cross-validation framework for hyper-parameter selection using $D.train$.

Algorithm 3.2 generalizes the above approach using a $k$-fold cross-validation framework for hyper-parameter selection. At the $i^{\text{th}}$ run of cross-validation, the data in the $i^{\text{th}}$ fold is used as $D.val(i)$ for validation (Step 4), while the remainder of the data in $D.train$ is used as $D.tr(i)$ for training (Step 5). Then for every choice of hyper-parameter value $p_i$, a model is learned on $D.tr(i)$ (Step 7), which is applied on $D.val(i)$ to compute its validation error (Step 8). This is used to compute the validation error rate corresponding to models learning using $p_i$ over all the folds (Step 11). The hyper-parameter value $p^*$ that provides the lowest validation error rate (Step 12) is now used to learn the final model $m^*$ on the entire training set $D.train$ (Step 13). Hence, at the

**Algorithm 3.2** Procedure `model-select`($k$, $\mathcal{P}$, $D.train$)

1: $N_{train} = |D.train|$ {Size of $D.train$.}
2: Divide $D.train$ into $k$ partitions, $D.train_1$ to $D.train_k$.
3: **for** each run $i = 1$ to $k$ **do**
4: $D.val(i) = D.train_i$. {Partition used for validation.}
5: $D.tr(i) = D.train \setminus D.train_i$. {Partitions used for training.}
6: **for** each parameter $p \in \mathcal{P}$ **do**
7: $m =$ `model-train`$(p, D.tr(i))$. {Train model}
8: $err_{sum}(p, i) =$ `model-test`$(m, D.val(i))$. {Sum of validation errors.}
9: **end for**
10: **end for**
11: $err_{val}(p) = \sum_i^k err_{sum}(p, i)/N_{train}$. {Compute validation error rate.}
12: $p^* = \text{argmin}_p\ err_{val}(p)$. {Select best hyper-parameter value.}
13: $m^* =$ `model-train`$(p^*, D.train)$. {Learn final model on $D.train$}
14: return $(p^*, m^*)$.

end of this algorithm, we obtain the best choice of the hyper-parameter value as well as the final classification model (Step 14), both of which are obtained by making an effective use of every data instance in $D.train$.

### 3.7.2 Nested Cross-Validation

The approach of the previous section provides a way to effectively use all the instances in $D.train$ to learn a classification model when hyper-parameter selection is required. This approach can be applied over the entire data set $D$ to learn the final classification model. However, applying Algorithm 3.2 on $D$ would only return the final classification model $m^*$ but not an estimate of its generalization performance, $err_{test}$. Recall that the validation error rates used in Algorithm 3.2 cannot be used as estimates of generalization performance, since they are used to guide the selection of the final model $m^*$. However, to compute $err_{test}$, we can again use a cross-validation framework for evaluating the performance on the entire data set $D$, as described originally in Section 3.6.2. In this approach, $D$ is partitioned into $D.train$ (for training) and $D.test$ (for testing) at every run of cross-validation. When hyper-parameters are involved, we can use Algorithm 3.2 to train a model using $D.train$ at every run, thus "internally" using cross-validation for model selection. This approach is called **nested cross-validation** or double cross-validation. Algorithm 3.3 describes the complete approach for estimating $err_{test}$ using nested cross-validation in the presence of hyper-parameters.

As an illustration of this approach, see Figure 3.35 where the labeled set $D$ is partitioned into $D.train$ and $D.test$, using a 3-fold cross-validation method.

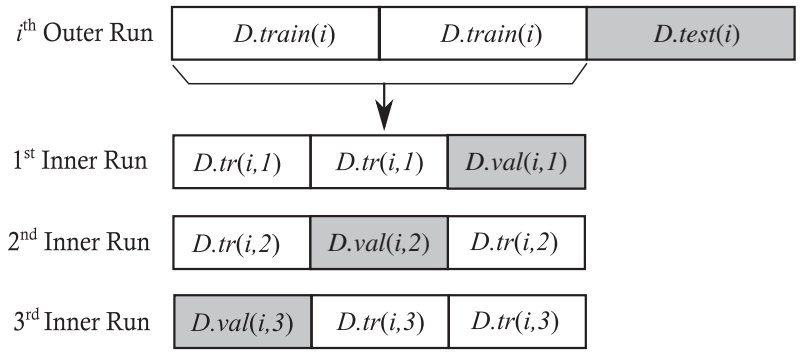



**Figure 3.35.** Example demonstrating 3-fold nested cross-validation for computing $err_{test}$.

At the $i^{\text{th}}$ run of this method, one of the folds is used as the test set, $D.test(i)$, while the remaining two folds are used as the training set, $D.train(i)$. This is represented in Figure 3.35 as the $i^{\text{th}}$ "outer" run. In order to select a model using $D.train(i)$, we again use an "inner" 3-fold cross-validation framework that partitions $D.train(i)$ into $D.tr$ and $D.val$ at every one of the three inner runs (iterations). As described in Section 3.7, we can use the inner cross-validation framework to select the best hyper-parameter value $p^*(i)$ as well as its resulting classification model $m^*(i)$ learned over $D.train(i)$. We can then apply $m^*(i)$ on $D.test(i)$ to obtain the test error at the $i^{\text{th}}$ outer run. By repeating this process for every outer run, we can compute the average test error rate, $err_{test}$, over the entire labeled set $D$. Note that in the above approach, the inner cross-validation framework is being used for model selection while the outer cross-validation framework is being used for model evaluation.

**Algorithm 3.3** The nested cross-validation approach for computing $err_{test}$.

1: Divide $D$ into $k$ partitions, $D_1$ to $D_k$.
2: **for** each outer run $i = 1$ to $k$ **do**
3: $D.test(i) = D_i$. {Partition used for testing.}
4: $D.train(i) = D \setminus D_i$. {Partitions used for model selection.}
5: $(p^*(i), m^*(i)) =$ `model-select`$(k, \mathcal{P}, D.train(i))$. {Inner cross-validation.}
6: $err_{sum}(i) =$ `model-test`$(m^*(i), D.test(i))$. {Sum of test errors.}
7: **end for**
8: $err_{test} = \sum_i^k \ err_{sum}(i)/N$. {Compute test error rate.}